

# **vLLM: Easy, Fast, and Cheap LLM Serving for Everyone**

Kaichao You, Tsinghua University

# The era of Large Language Models (LLMs)

Chat



Program



Search



...

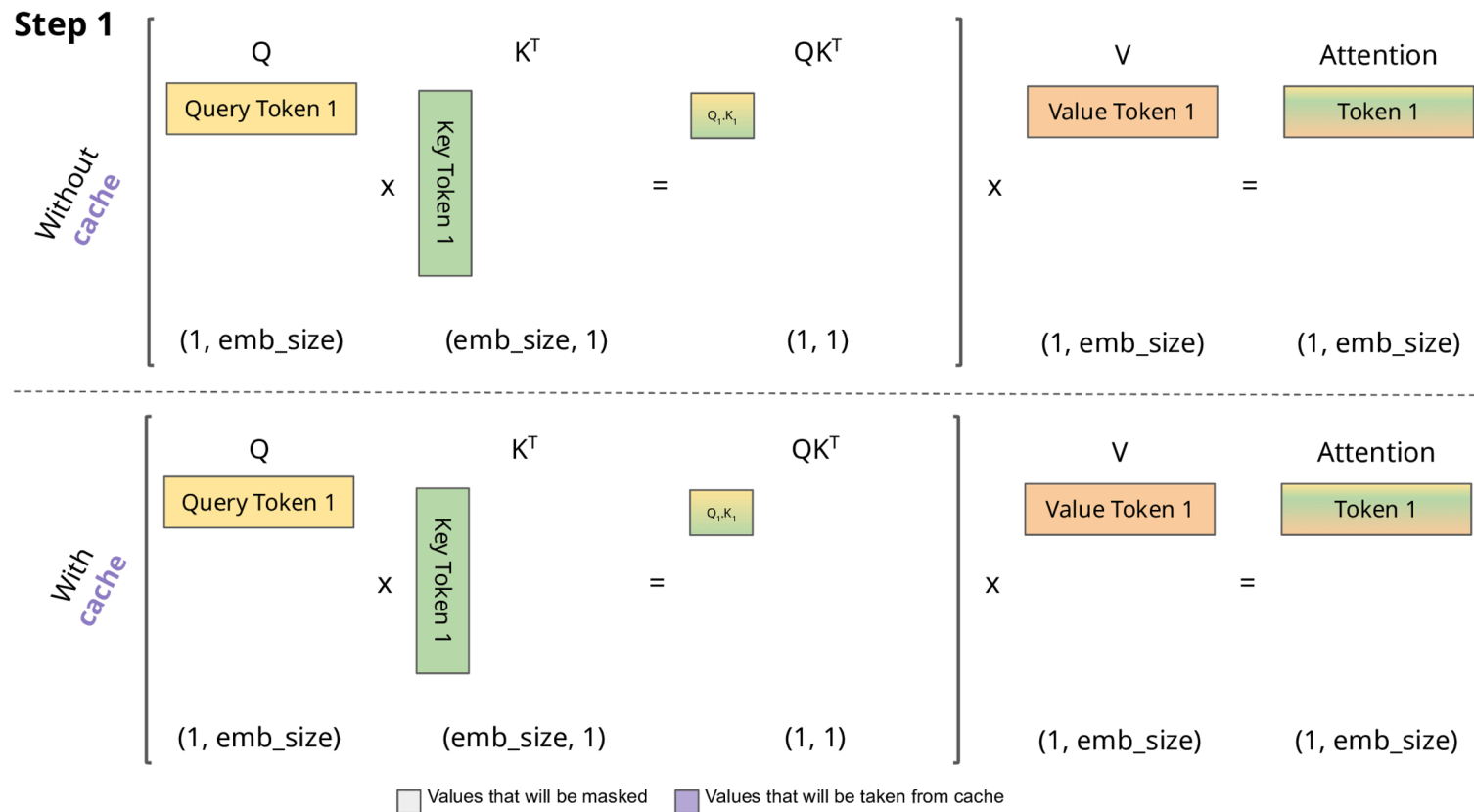
## Serving LLMs was (surprisingly) slow and expensive

- A single A100 GPU can only serve  $< 1$  request per second
  - Moderate size of model (13B parameters) and input
- A ton of GPUs are required for production-scale LLM services



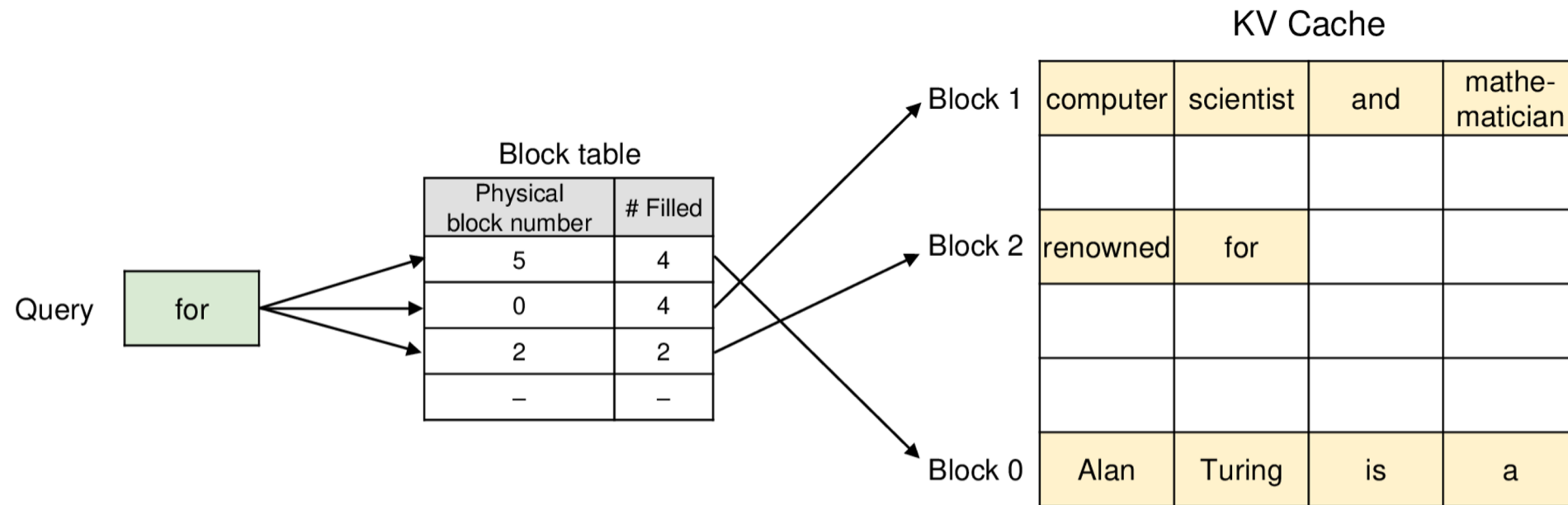
# KV Cache

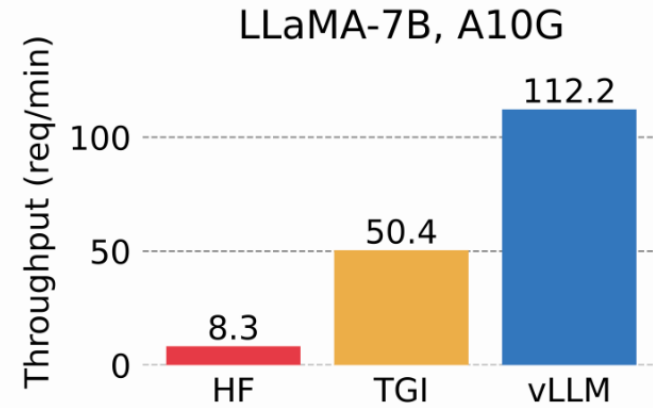
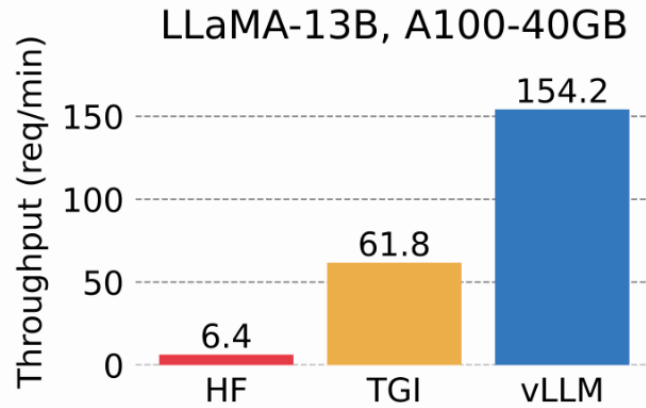
Caching Key and Value vectors in self-attention saves redundant computation and accelerates decoding



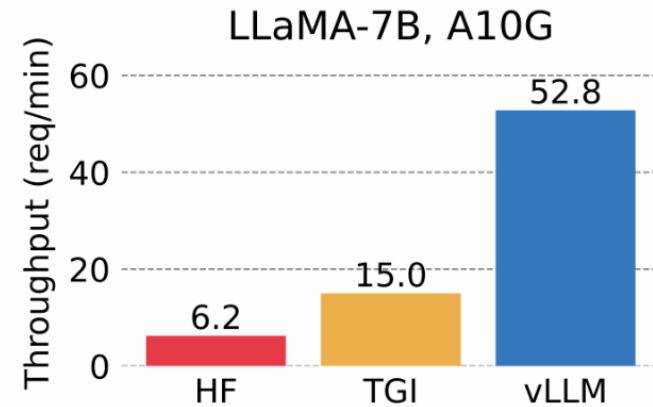
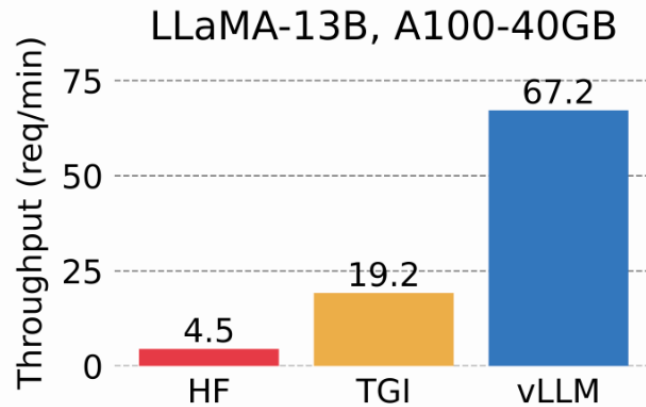
# PagedAttention

An attention algorithm that allows for storing continuous keys and values in non-contiguous memory space.





Serving throughput when each request asks for *one output completion*. vLLM achieves 14x - 24x higher throughput than HF and 2.2x - 2.5x higher throughput than TGI.



Serving throughput when each request asks for *three parallel output completions*. vLLM achieves 8.5x - 15x higher throughput than HF and 3.3x - 3.5x higher throughput than TGI.

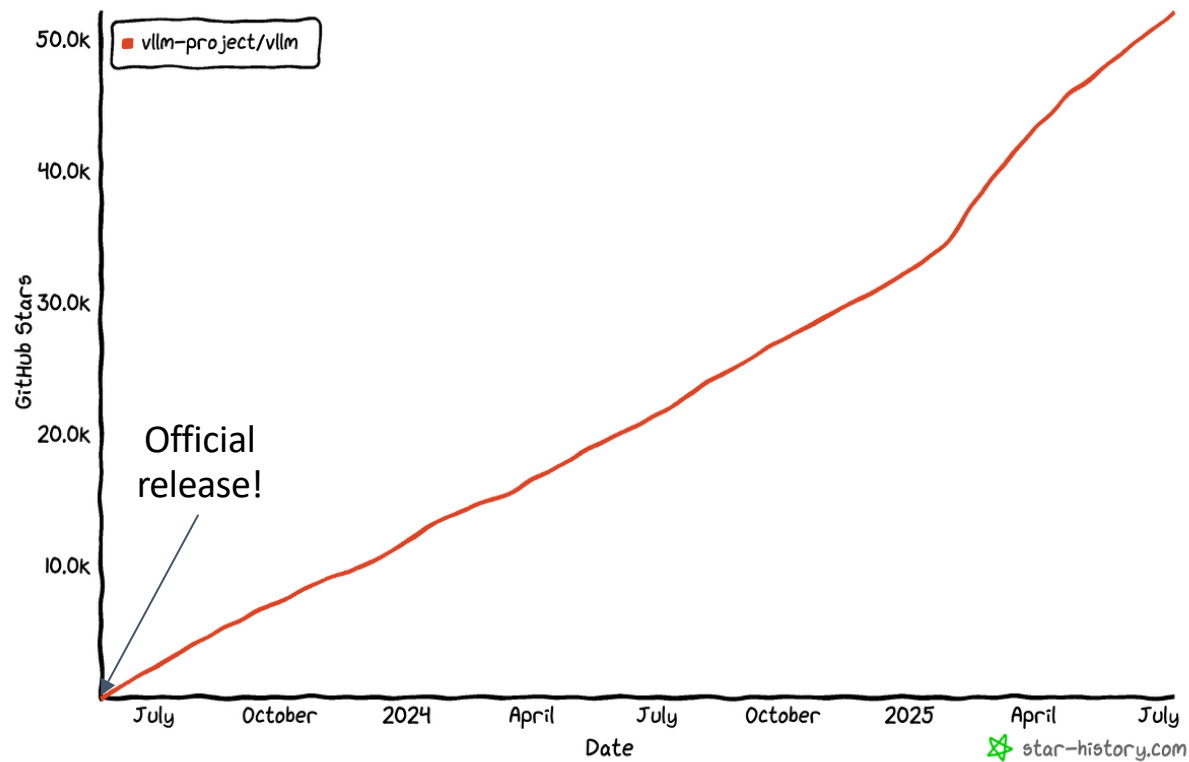


\$ pip install vllm



Star History

52.1K Stars



# VLLM 's Goal

Build the **fastest** and  
**easiest-to-use** open-source  
LLM inference & serving engine

## vLLM API (1): LLM class

A Python interface for offline batched inference

```
from vllm import LLM

# Example prompts.
prompts = ["Hello, my name is", "The capital of France is"]
# Create an LLM with HF model name.
llm = LLM(model="meta-llama/Meta-Llama-3.1-8B")
# Generate texts from the prompts.
outputs = llm.generate(prompts)
```



# vLLM API (1): LLM class

## Quick Start

- Try PyTorch workflow with a few LoC based on LLM API

```
# Import LLM API
from tensorrt_llm import LLM

# Create a LLM object
llm = LLM(model="./Llama-3.1-8B-instruct")

# Prepare prompts
prompts = [
    "Hi, pls tell me something about reasoning model",
    "Hi, pls tell me something about TensorRT-LLM"
]

# Generate output
output = llm.generate(prompts)
```

TensorRT workflow



```
# Import LLM API
from tensorrt_llm.torch import LLM

# Create a LLM object
llm = LLM(model="./Llama-3.1-8B-instruct")

# Prepare prompts
prompts = [
    "Hi, pls tell me something about reasoning model",
    "Hi, pls tell me something about TensorRT-LLM"
]

# Generate output
output = llm.generate(prompts)
```

PyTorch workflow

\* the credit for the LLM API idea goes to the vLLM team

- More examples with additional parameters are available in [examples/pytorch/quickstart\\_advanced.py](#)
  - **LLMArgs**: Model path, tokenizer, tensor parallelism, quantization, ...
    - All args can also be directly passed to LLM() as kwargs
  - **PyTorchConfig**: Additional configs for PyTorch such as CUDA graph and selection of different attention backend, etc.

## vLLM API (2): OpenAI-compatible server

A FastAPI-based server for online serving

### Server

```
$ vllm serve meta-llama/Meta-Llama-3.1-8B
```

### Client

```
$ curl http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Meta-Llama-3.1-8B",
    "prompt": "San Francisco is a",
    "max_tokens": 7,
    "temperature": 0
  }'
```

\* Welcome to **Claude Code**!

## vLLM API (3): Embeddable LLMEngine

A Python library with the full power of vLLM in your framework

```
from vllm.engine.arg_utils import AsyncEngineArgs
from vllm.engine.async_llm_engine import AsyncLLMEngine
from vllm.sampling_params import SamplingParams

engine_args = AsyncEngineArgs.from_cli_args(args)
engine = AsyncLLMEngine.from_engine_args(engine_args)

results_generator = engine.generate(prompt, sampling_params, request_id)

async for request_output in results_generator:
    for output in request_output.outputs:
        yield output.text
```



**NVIDIA**

TRITON INFERENCE SERVER

## Broad Model Support

- Transformer-like LLMs (e.g., Llama)
- Mixture-of-Expert LLMs (e.g., DeepSeek)
- Multi-modal LLMs (e.g., LLaVA)
- State-Space Models (e.g., Jamba)
- Linear-Attention Models (e.g., Minimax-M1)
- Embedding Models (e.g. E5-Mistral)
- Reward Models (e.g. Qwen2.5-Math-RM)

# Broad Model Support

vLLM supports **almost all popular open LLMs & VLMs!**

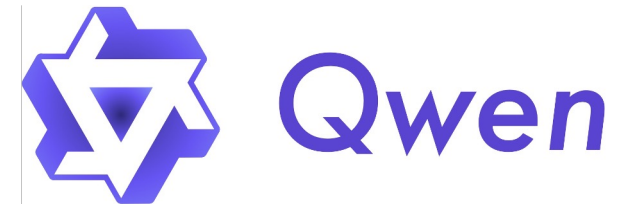
Llama 3.1 & 3.2 support on day 1



Contributed by model creators

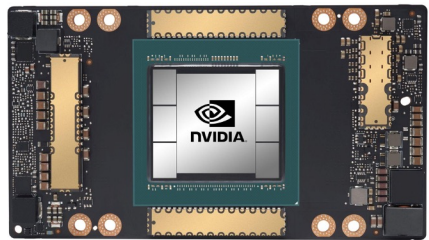


Pixtral 12B

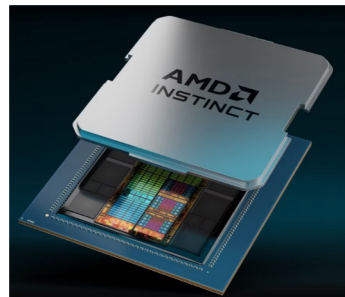


Qwen2-VL

# Diverse Hardware Support



NVIDIA GPU



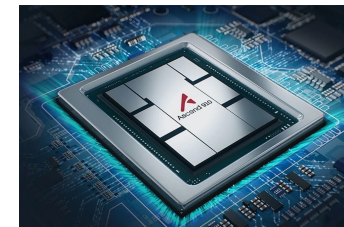
AMD GPU



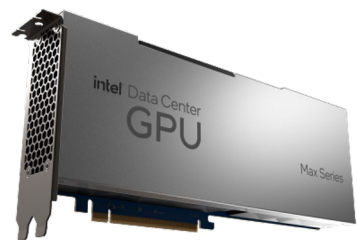
AWS Neuron



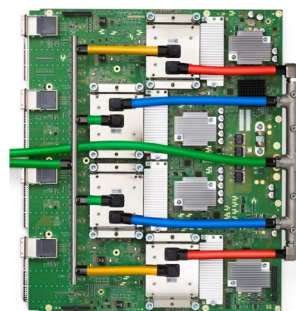
x86 CPU



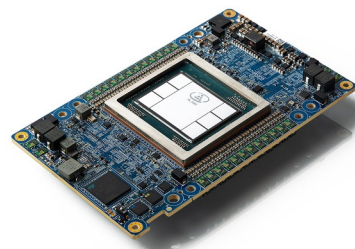
Huawei Ascend



Intel GPU



Google TPU



Intel Gaudi



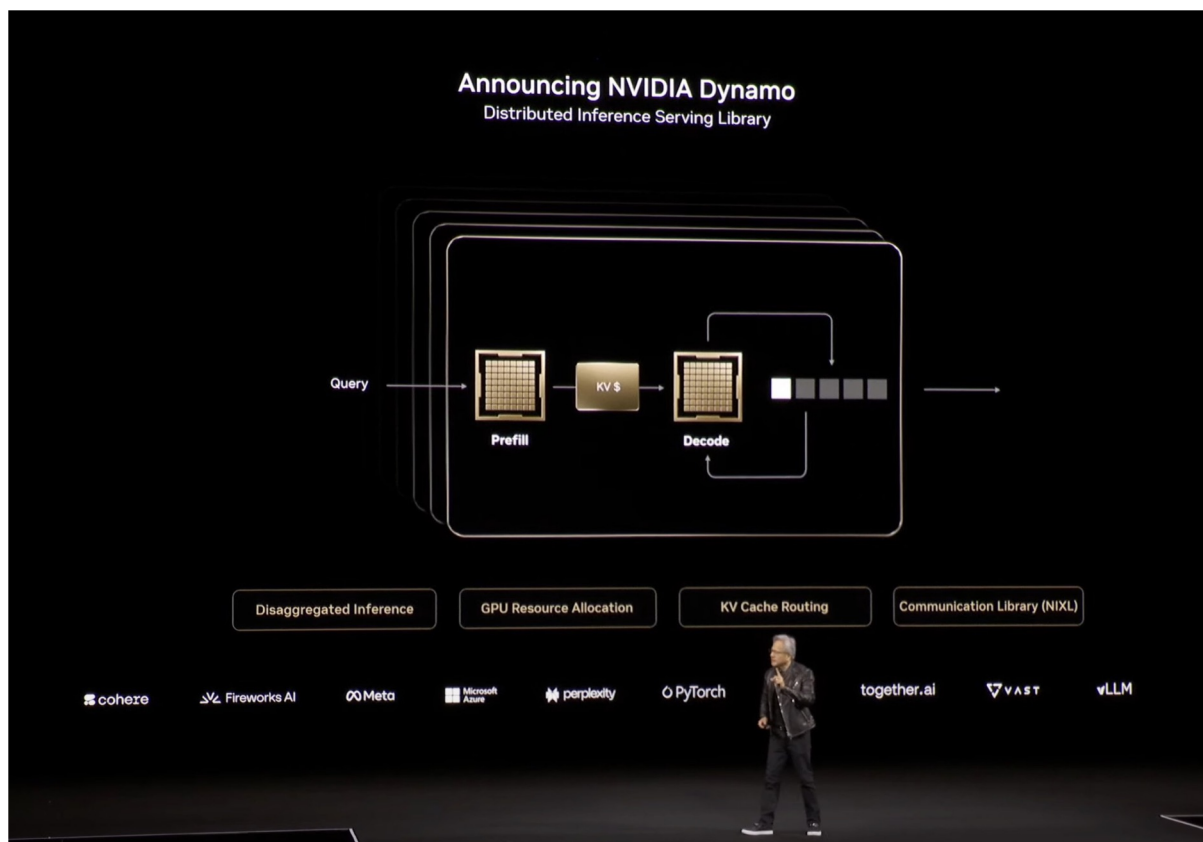
ARM CPU

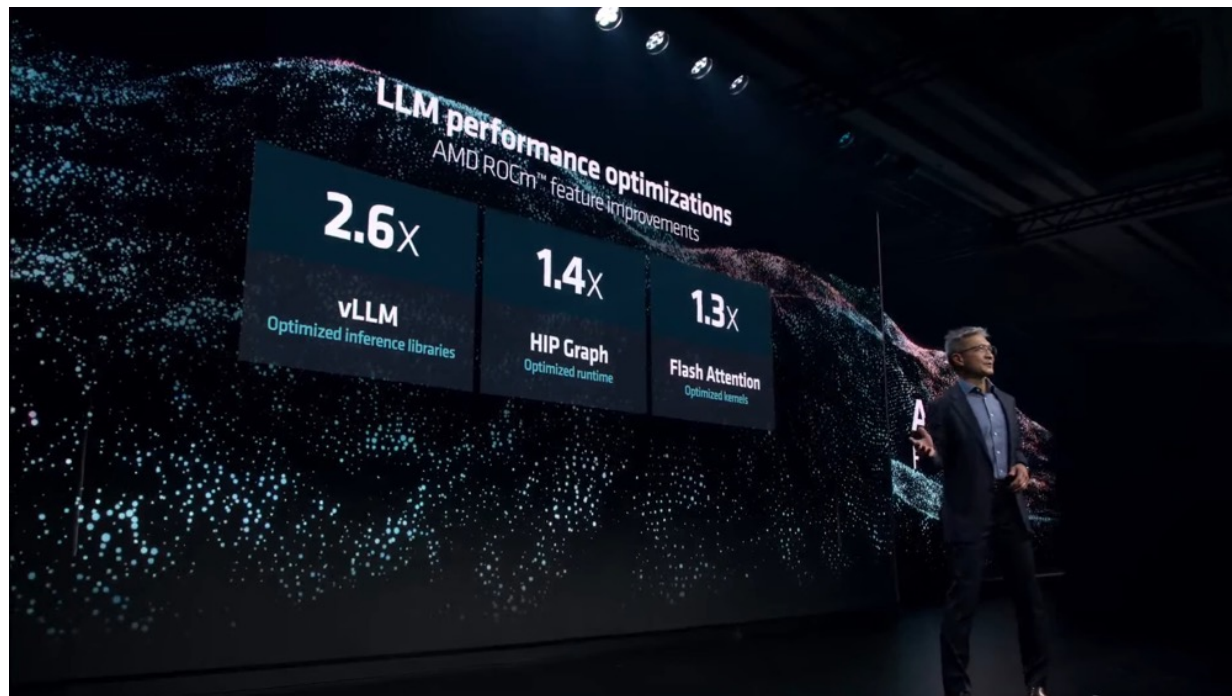


IBM Spyre

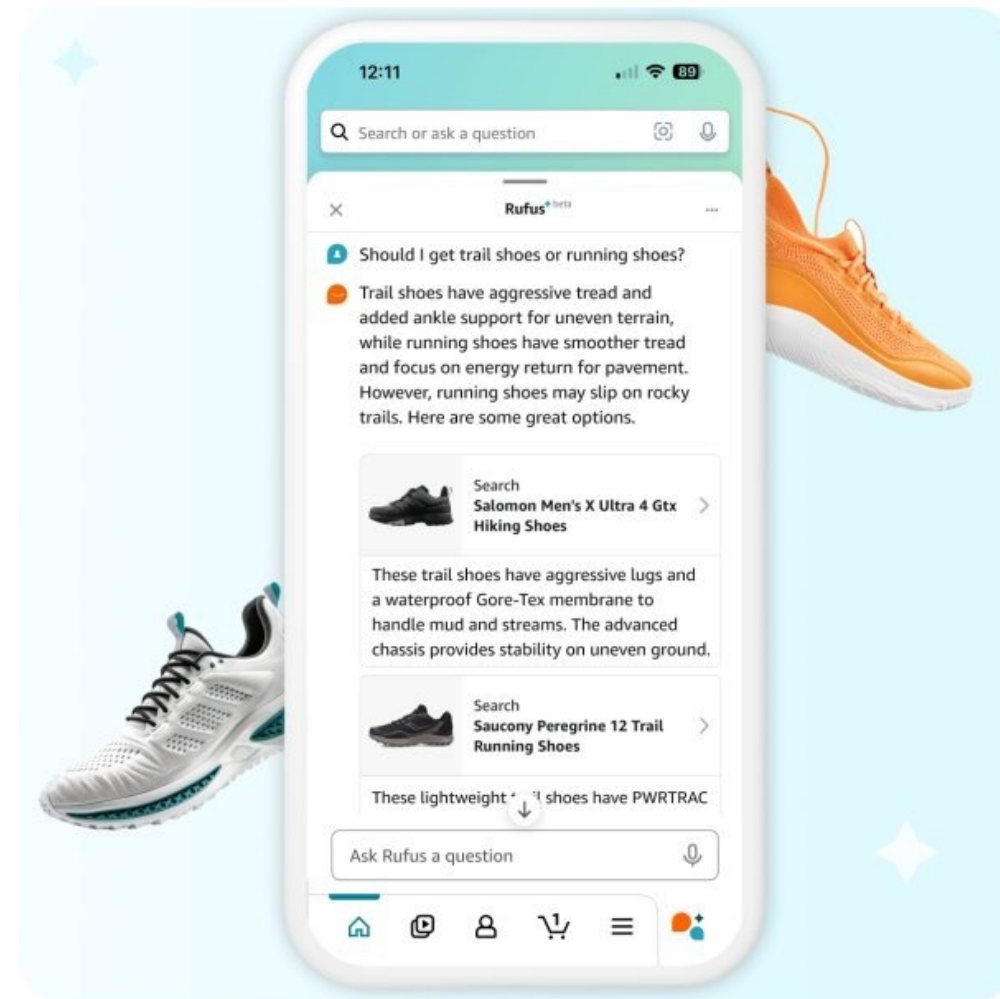
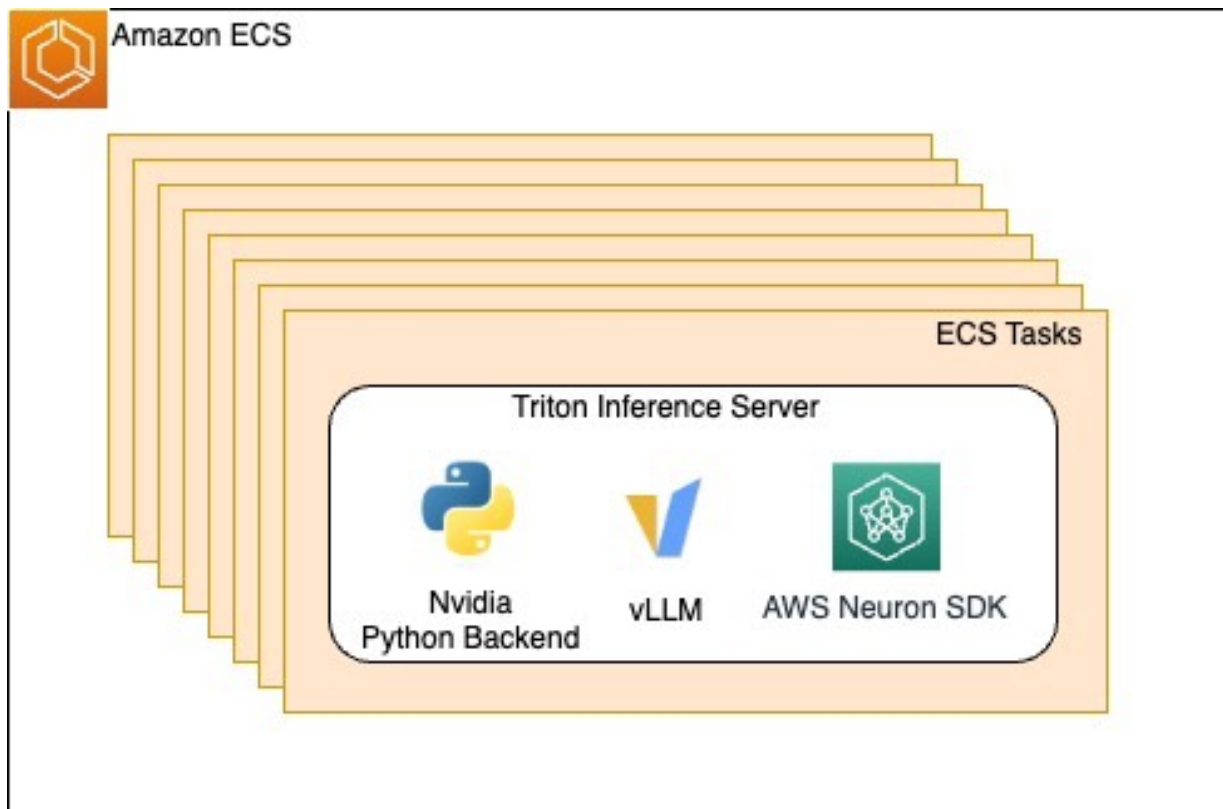
support in vllm repo

hardware plugin



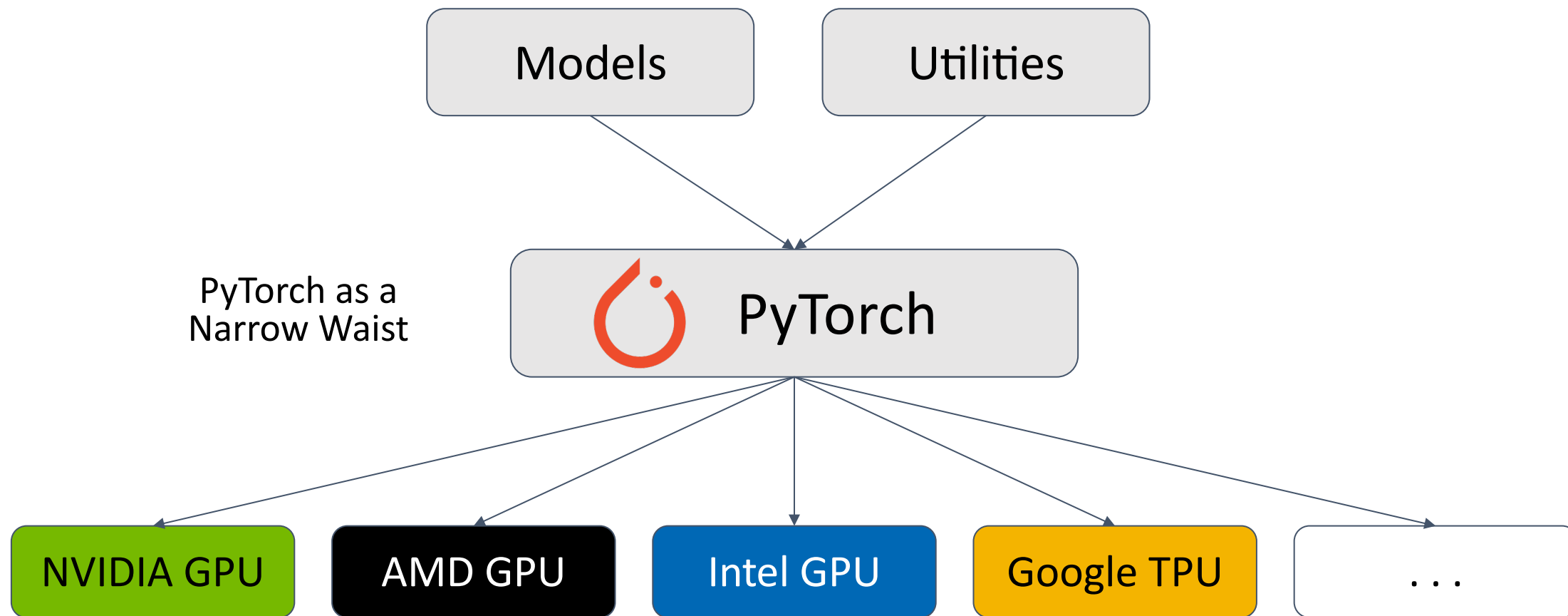


# Amazon Rufus Example



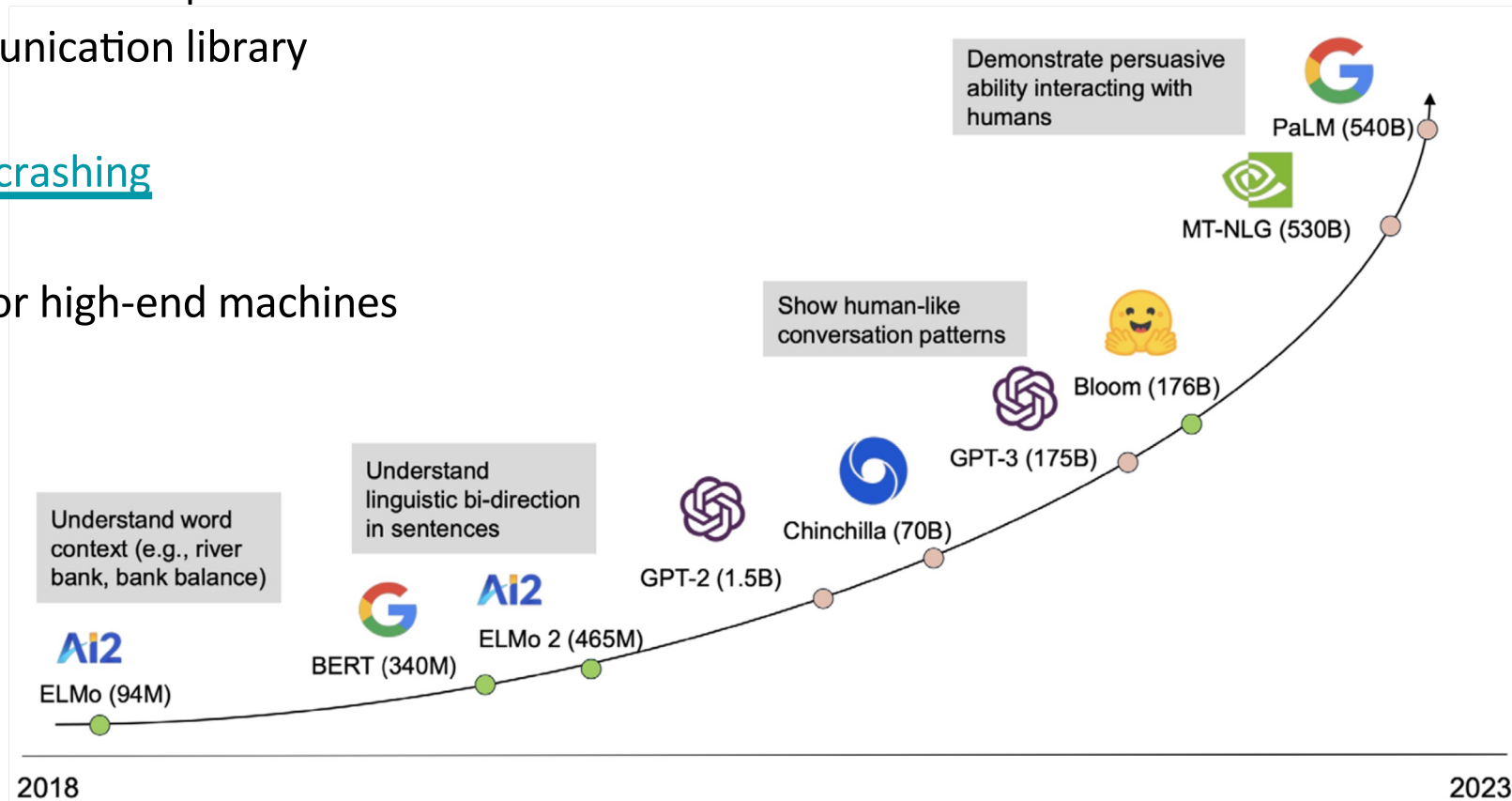
serving 3 million tokens per minute with over 80,000 Trainium and Inferentia chips, on Prime Day

# Diverse Hardware Support



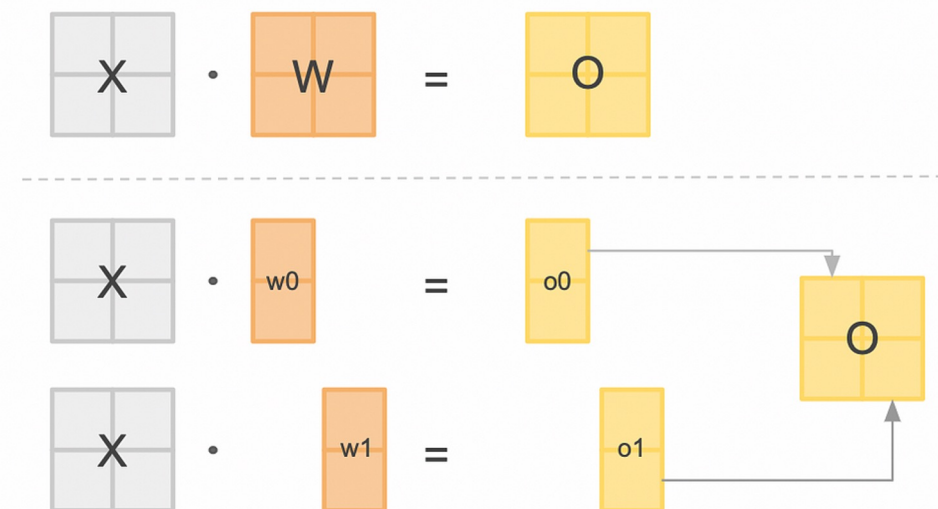
# vLLM handles distributed inference for you

- vLLM supports tensor parallelism from day one.
- Multi-GPU/multi-node inference is a **must** for large LLMs
- But cross-GPU/node communications are error-prone
  - hang/crash/versions of communication library
- vLLM takes care of it for you
  - [Debugging guide for hanging/crashing](#)
  - set proper NCCL flags
  - Optimized custom allreduce for high-end machines



## Tensor parallelism (in vLLM)

- Partition the weights in the linear layers
- 👍 Reduce the serving latency if the network is fast
- 👍 KV Cache can also be partitioned
- 👎 Heavy communication across TP shards. Often require NVLink to work effectively.



# Sequence parallelism (in vLLM)

- An additional optimization for tensor parallel
- 👍 Change all-reduce to AG/RS, faster layernorm, smaller memory footprint
- 👎 Need to change the model definition for every model

```
vllm serve meta-llama/Llama-3.2-1B-Instruct \
--tensor-parallel-size 2 \
--pipeline-parallel-size 2 \
--distributed-executor-backend mp \
-O '{"level": 3, "compile_sizes": [4, 8, 16], "custom_ops": ["+rms_norm"],
"splitting_ops": [], "pass_config": {"enable_sequence_parallelism": true}}'
```

Implemented as a compilation pass, applies to all models!

## Sequence Parallelism of Layernorm, Dropout

- Idea: Layernorm and dropout activations can be parallelized across the sequence dimension, so we distribute those across tensor parallel ranks.

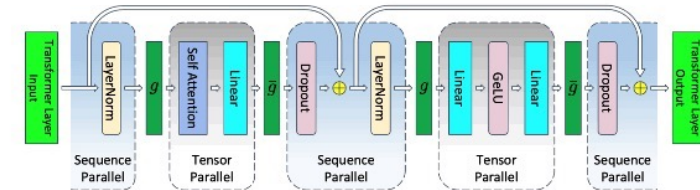


Figure 5: Transformer layer with tensor and sequence parallelism.  $g$  and  $\bar{g}$  are conjugate.  $g$  is all-gather in the forward pass and reduce-scatter in the backward pass.  $\bar{g}$  is reduce-scatter in forward pass and all-gather in backward pass.

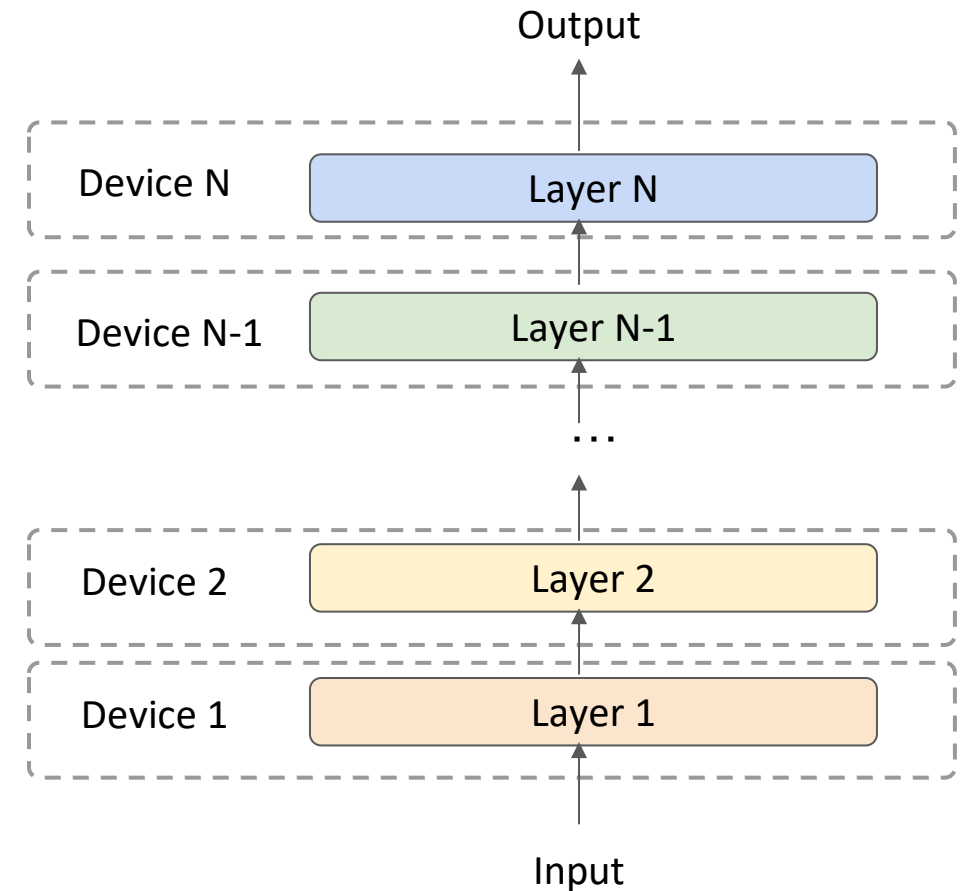
$$TP = t - sbh(10 + 24/t + 5as/ht)$$

$$TP = t + \text{Sequence Parallel} - sbh(34/t + 5as/ht)$$

70

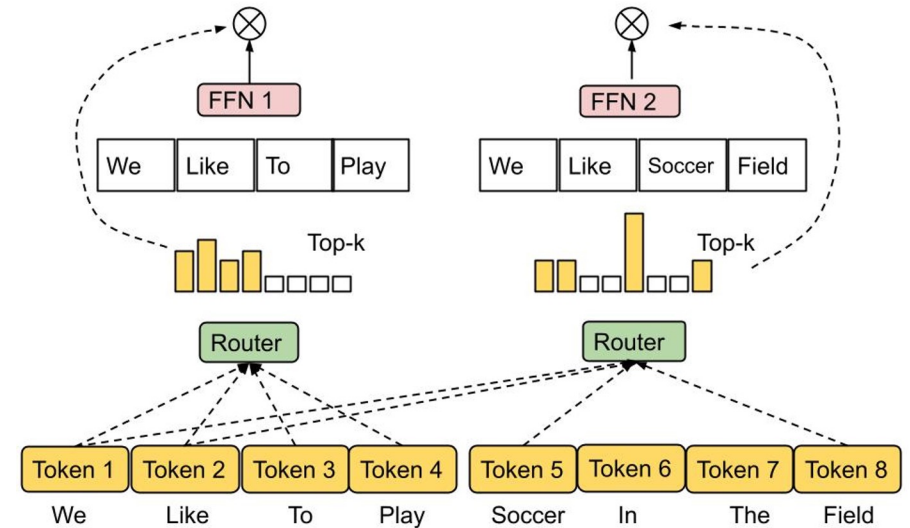
# Pipeline parallelism (in vLLM)

- Distribute the layers to different GPUs and pipeline the execution across devices
- 👍 Lowest communication overhead
- 👎 Increased latency
- 👎 Throughput bottlenecked by the slowest stage
- 👎 Need multiple batches to keep pipeline high-utilization, while multiple batches multiplies the KV cache as well



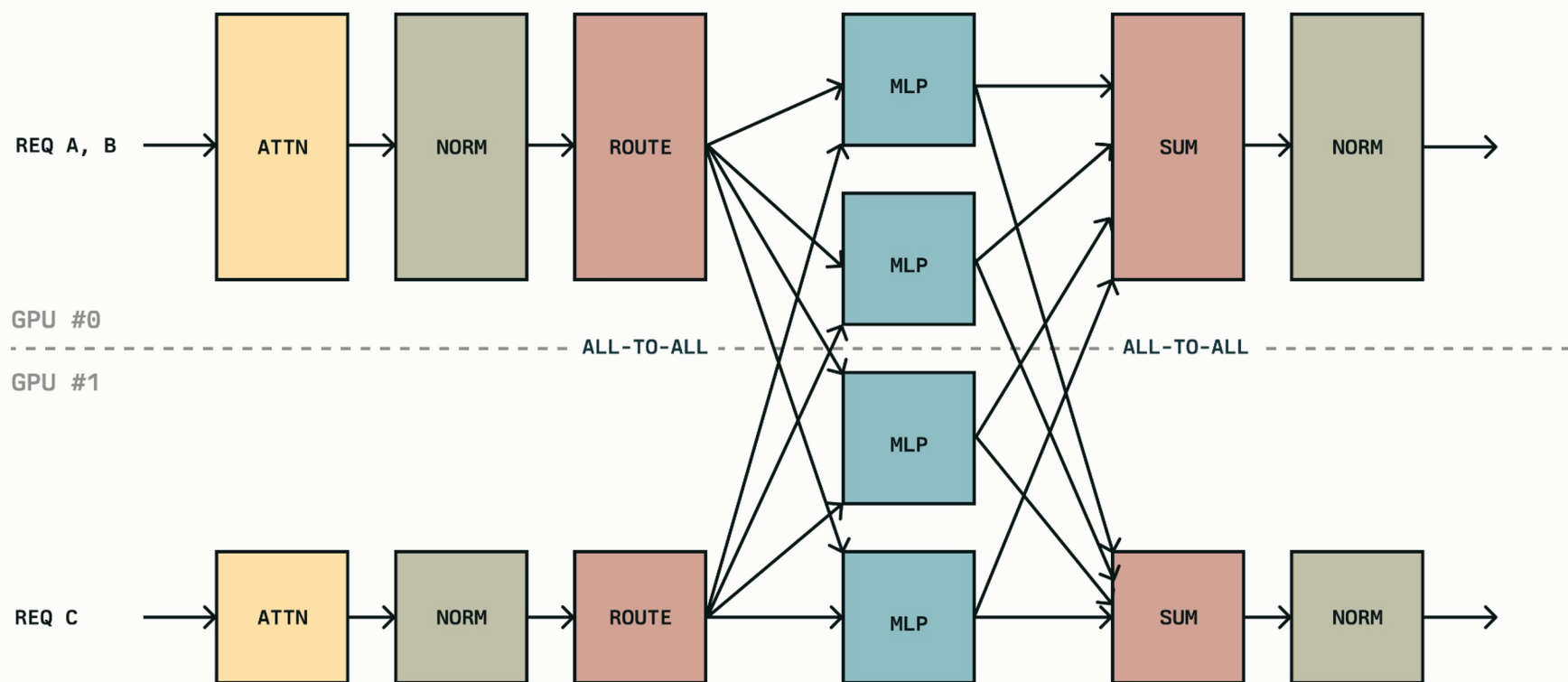
# Expert parallelism (in vLLM)

- Mixture of Expert: A set of linear layers (experts) where each token only pick a subset to run on
- Distribute different experts to different nodes
- 👍 Friendly for GPU kernels
- 👍 Smaller communication than TP
- 👎 Only apply for MoE layers. Attention needs DP.
- 👎 Different experts have different loads, which needs to be balanced



# Expert parallelism + Data parallelism

## EXPERT PARALLEL + DATA PARALLEL

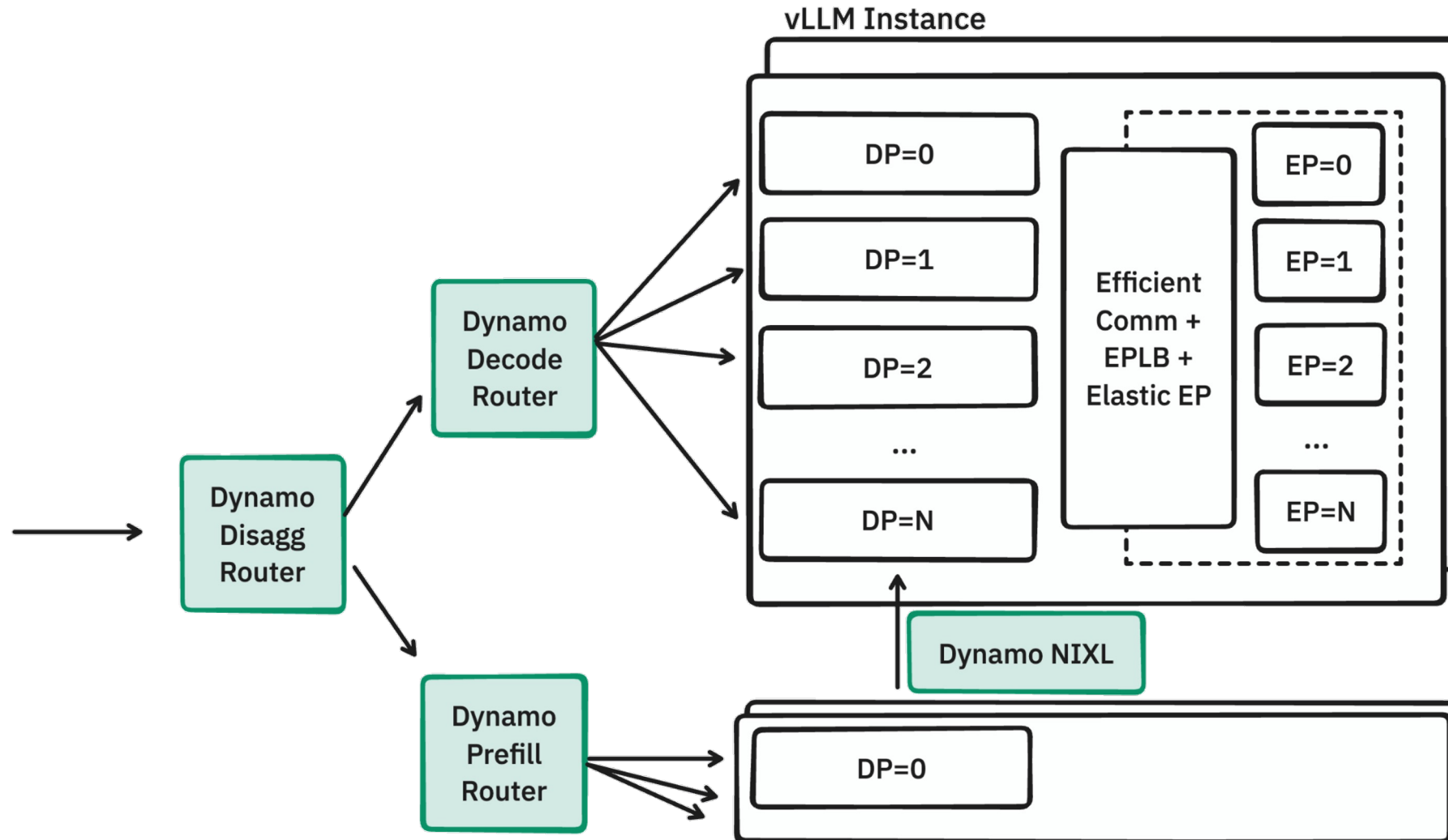


```
--enable-eplb  
--num-redundant-experts 32  
--eplb-window-size 1000  
--eplb-step-interval 3000
```

# Prefill-Decode Disaggregation

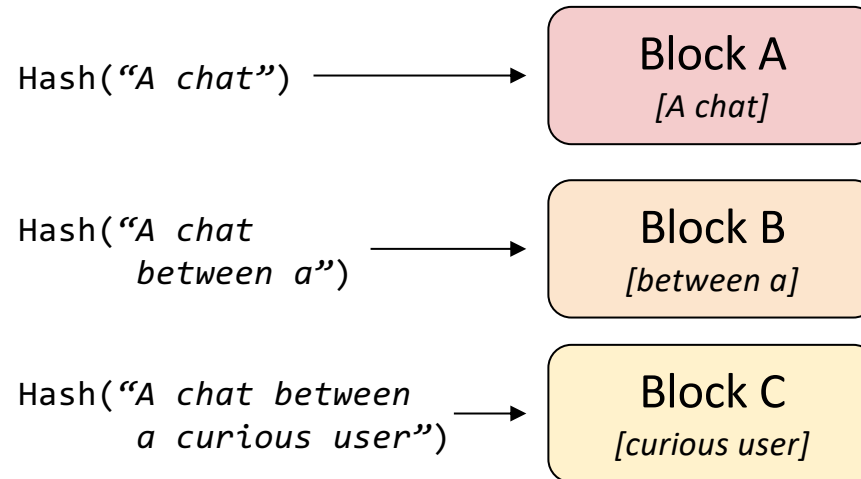
- Different parallelization strategies have different characteristics in computation/memory/communication requirements
- Different stages of LLM inference have different characteristics as well:
  - Prefill: typically compute bound
  - Decode: typically memory bound and require large KV cache memory
- **Disaggregation:** Have different setups for prefill and decode. Transfer the KV cache between them.
- *Side benefit:* Separate concern on time-to-first-token/time-per-output-token

# Architecture



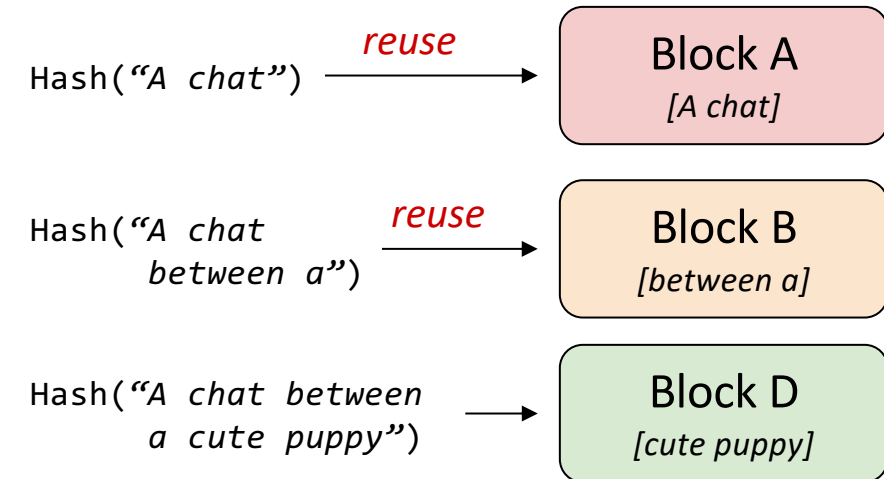
# Hash-Based Automatic Prefix Caching

**Request:** *A chat between a curious user ...*



**Request KV  
Blocks:**

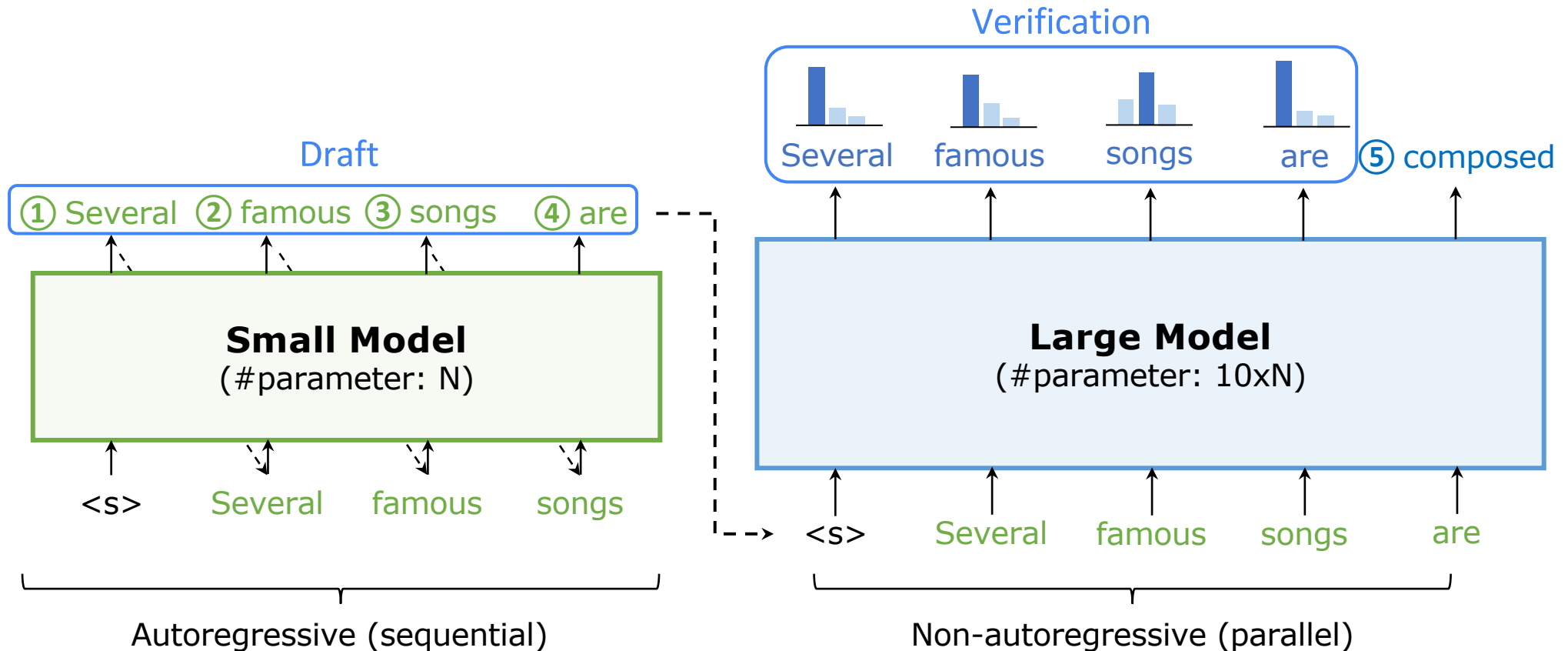
*A chat between a **cute puppy** ...*



# Speculative Decoding

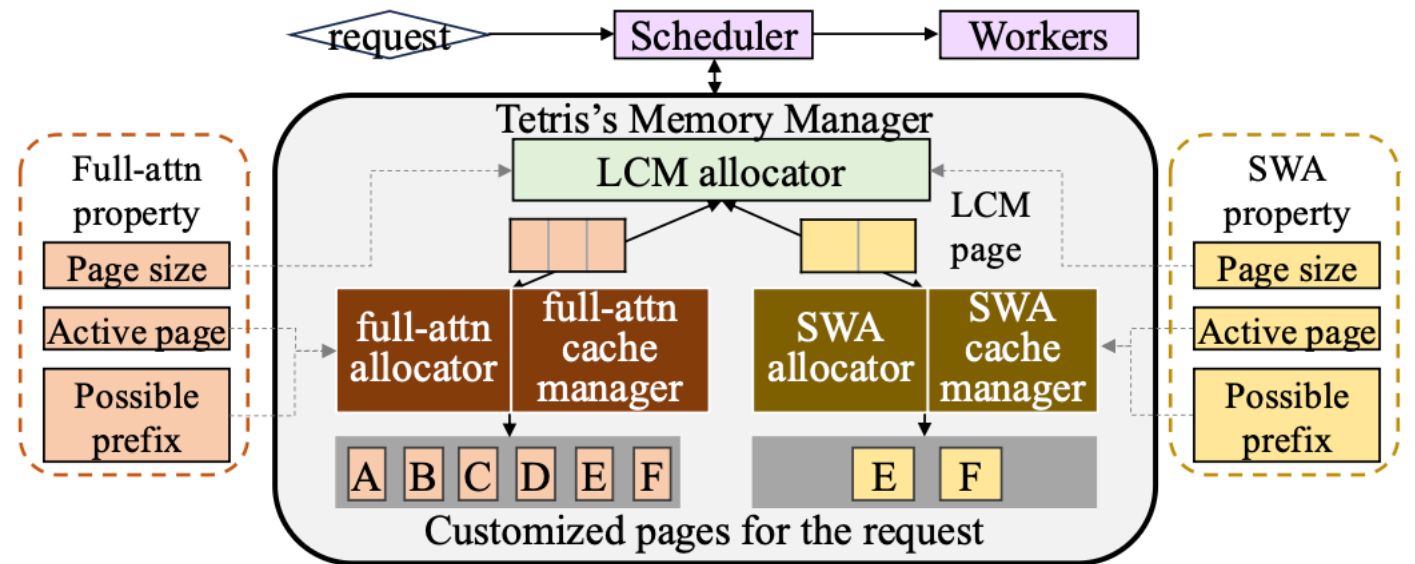
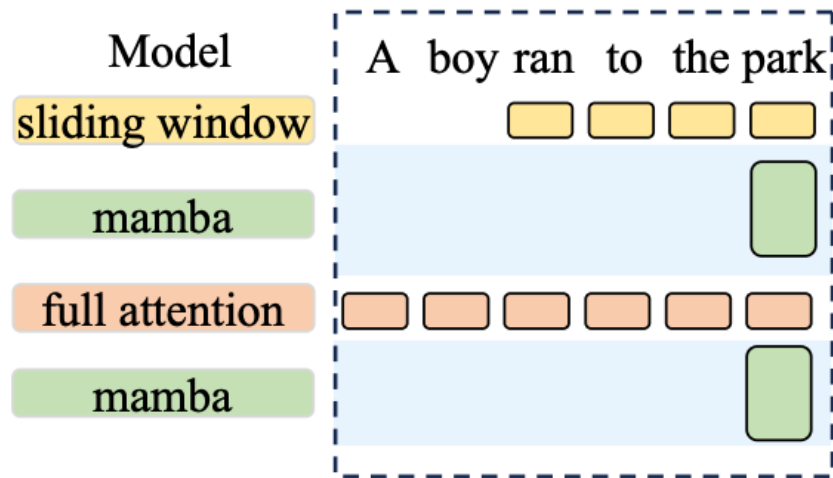
Small model writes a **draft** → Large model **verifies** it

**Much faster:** verifying 5 tokens takes similar time as generating 1 token



# Hybrid Allocator

Long context models usually adopt a hybrid architecture, with interleaving full attention and sliding window/linear attention.  
How to manage the heterogeneity in memory management?



Supported and turned on by default.

# Sleep Mode

Why you'll want this

- (Serverless) Hot-swap new checkpoints on the same card
- (Resource sharing) Rotate multiple LLMs on one GPU (batch jobs, micro-services, A/B tests)
- (Co-locate RL) RLHF / GRPO: sleep during training, wake for inference

Example:

# expose dev-only endpoints (internal net only!)

```
export VLLM_SERVER_DEV_MODE=1
```

```
vllm serve $MODEL --enable-sleep-mode
```

🛌 Power-nap

```
curl -X POST :8000/sleep -d '{"level":1}' # weights → CPU, KV cache gone
```

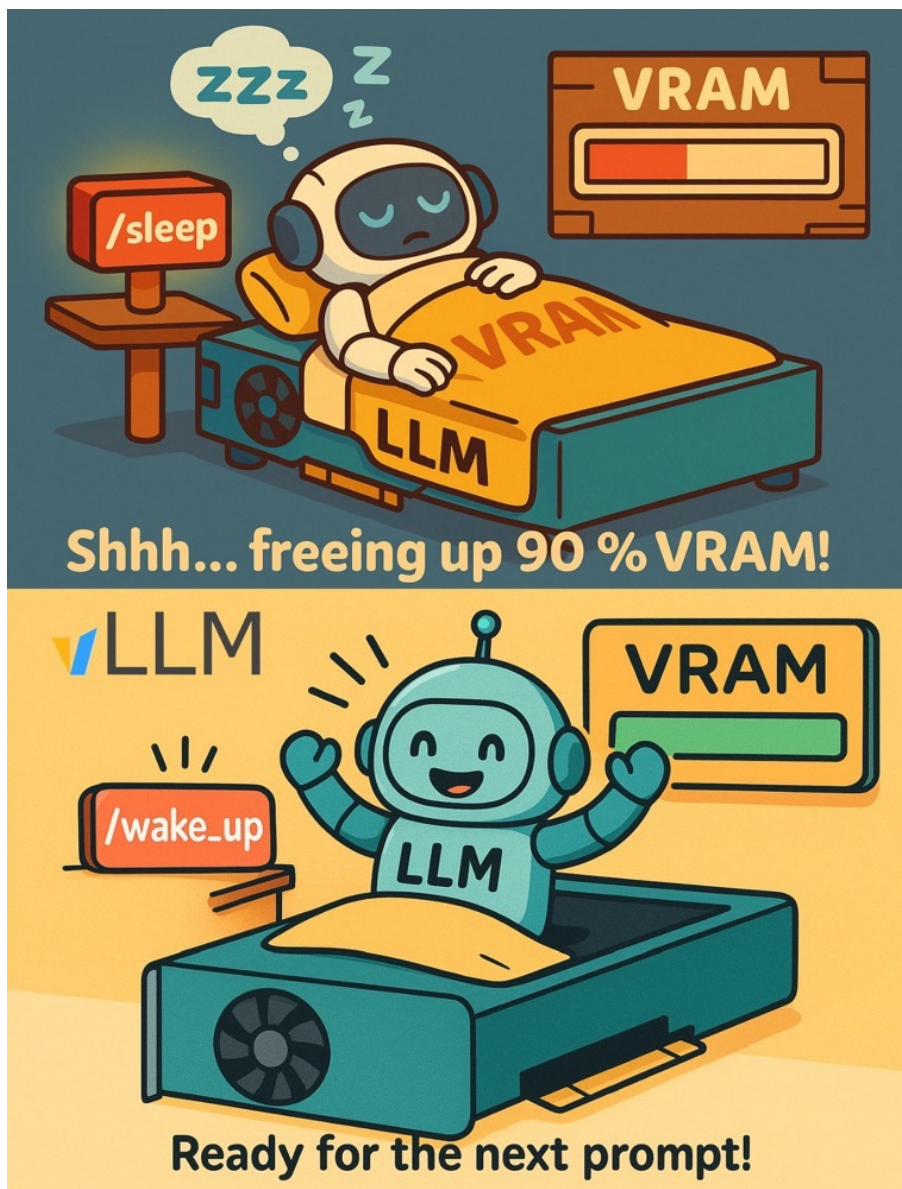
⚡ Wake up

```
curl -X POST :8000/wake_up # weights copy back, ready in secs
```

Heads-up

🔒 /sleep & /wake\_up skip auth—hide behind your own proxy

🧠 Level 1 keeps weights in host RAM; Level 2 frees everything but wakes slower



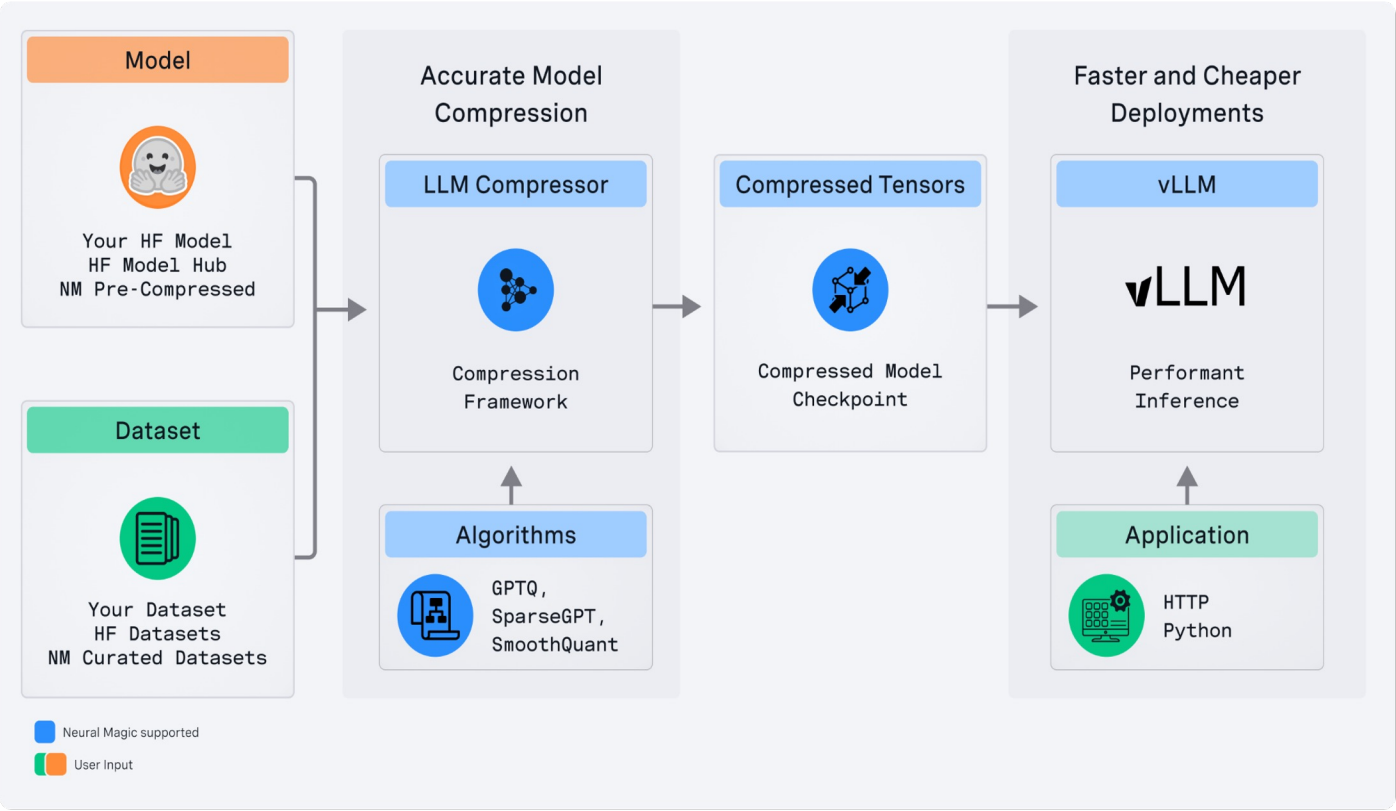
# LLM Quantization

- LLMs have 3 axes to quantize
  - Size: **Weights**  $\geq$  **KV cache**  $\ggg$  **Activation**
- 1. **Weight** quantization (e.g., FP8, INT8, GPTQ, AWQ)
  - Main benefits: Reduced storage & memory footprints
- 1. **KV cache** quantization (e.g., FP8)
  - Main benefits: Reduced KV cache storage & Faster attention
- 1. **Activation** quantization (e.g., FP8, INT8)
  - Main benefits: Faster GeMM & communication (in distributed inference)

# Quantization support in vLLM



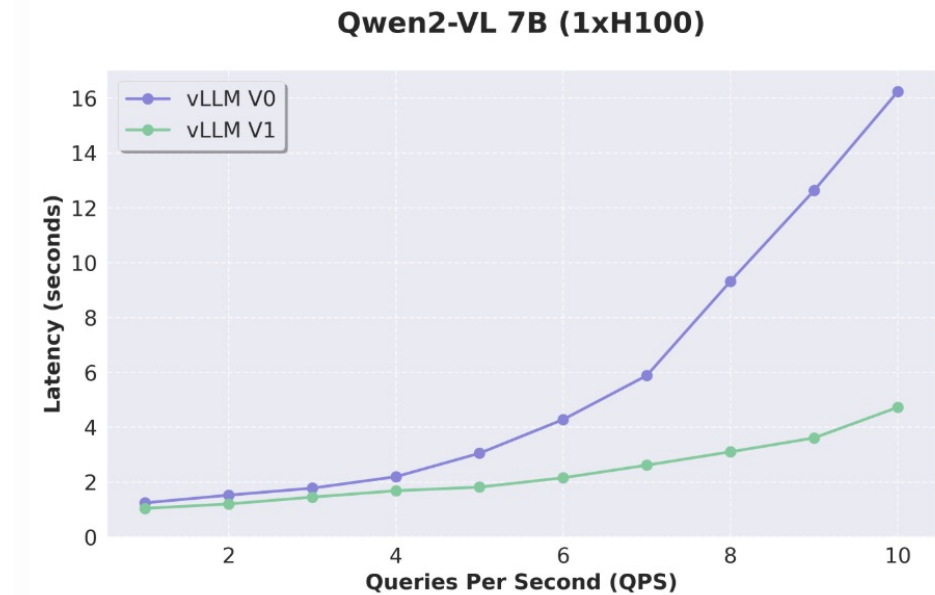
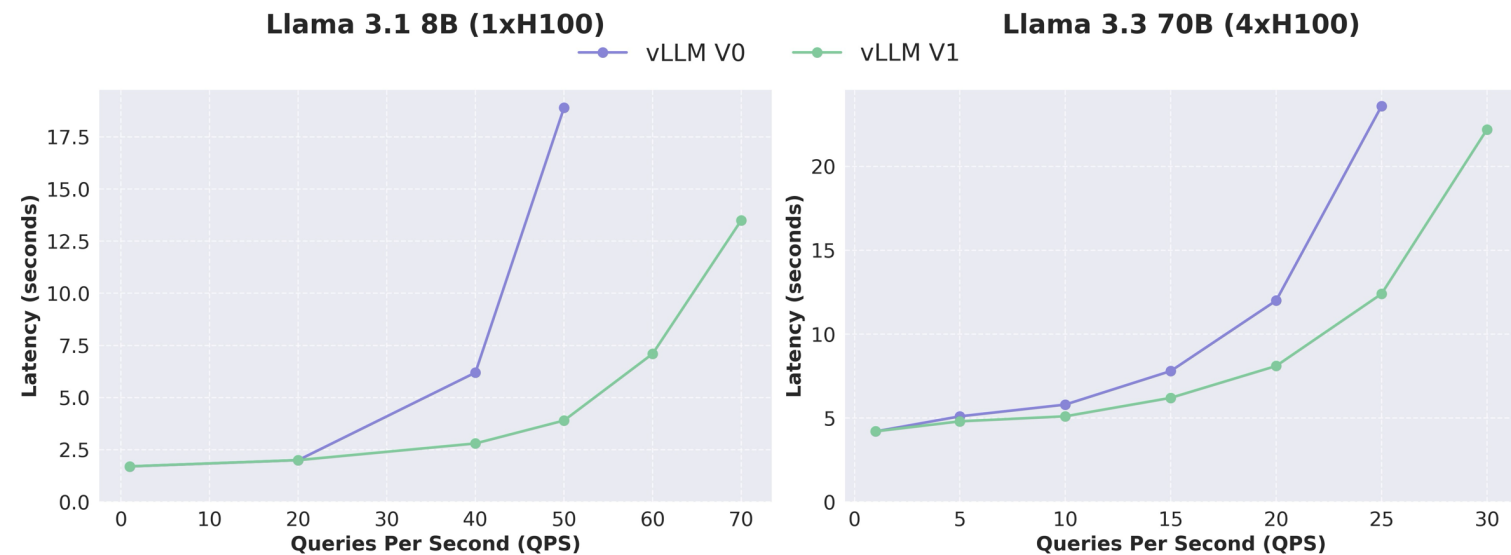
## LLM Compressor



- LLM Compressor library for quantizing the model weights
- vLLM implements highly tuned Cutlass kernels for quantized GeMMs

| Implementation           | Volta | Turing | Ampere | Ada | Hopper |
|--------------------------|-------|--------|--------|-----|--------|
| AWQ                      | ×     | ✓      | ✓      | ✓   | ✓      |
| GPTQ                     | ✓     | ✓      | ✓      | ✓   | ✓      |
| Marlin<br>(GPTQ/AWQ/FP8) | ×     | ×      | ✓      | ✓   | ✓      |
| INT8 (W8A8)              | ×     | ✓      | ✓      | ✓   | ✓      |
| FP8 (W8A8)               | ×     | ×      | ×      | ✓   | ✓      |
| AQLM                     | ✓     | ✓      | ✓      | ✓   | ✓      |
| bitsandbytes             | ✓     | ✓      | ✓      | ✓   | ✓      |
| DeepSpeedFP              | ✓     | ✓      | ✓      | ✓   | ✓      |
| GGUF                     | ✓     | ✓      | ✓      | ✓   | ✓      |

# vLLM V1: A major update

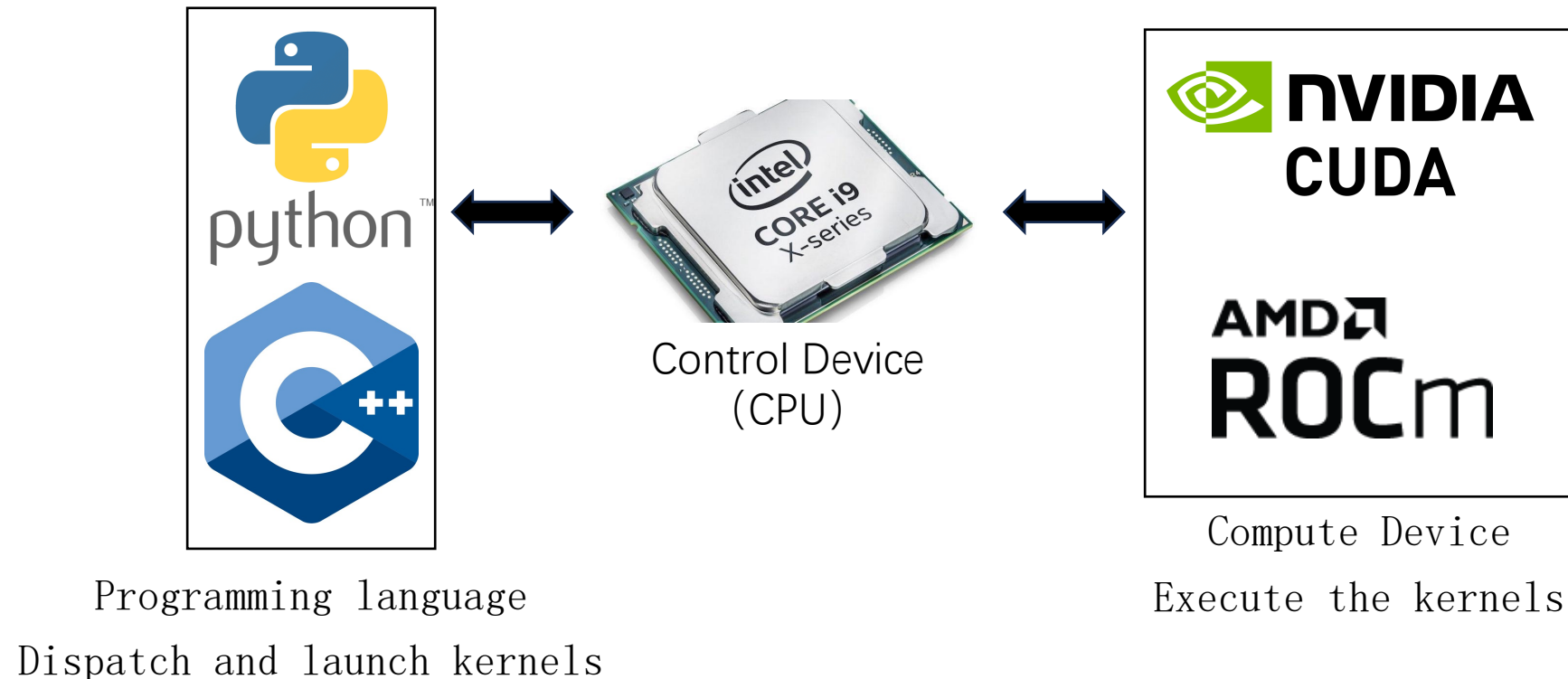


See <https://blog.vllm.ai/2025/01/27/v1-alpha-release.html>

# Bitter Lesson in LLM Inference

- The biggest lesson we've learned in vLLM

“To fully utilize the GPU, we need to **pay close attention to everything happening on the CPU** (i.e., **CPU overheads**)”



# CPU Overheads

- CPU overheads in an old version of vLLM



CPU time (29%): scheduling requests, preparing LLM inputs, organizing LLM outputs.

- Why so much overheads?
  - Python is slow
  - We didn't utilize multiple threads/processes efficiently in Python
  - PyTorch has performance pitfalls
  - Continuous batching makes input preparation complicated

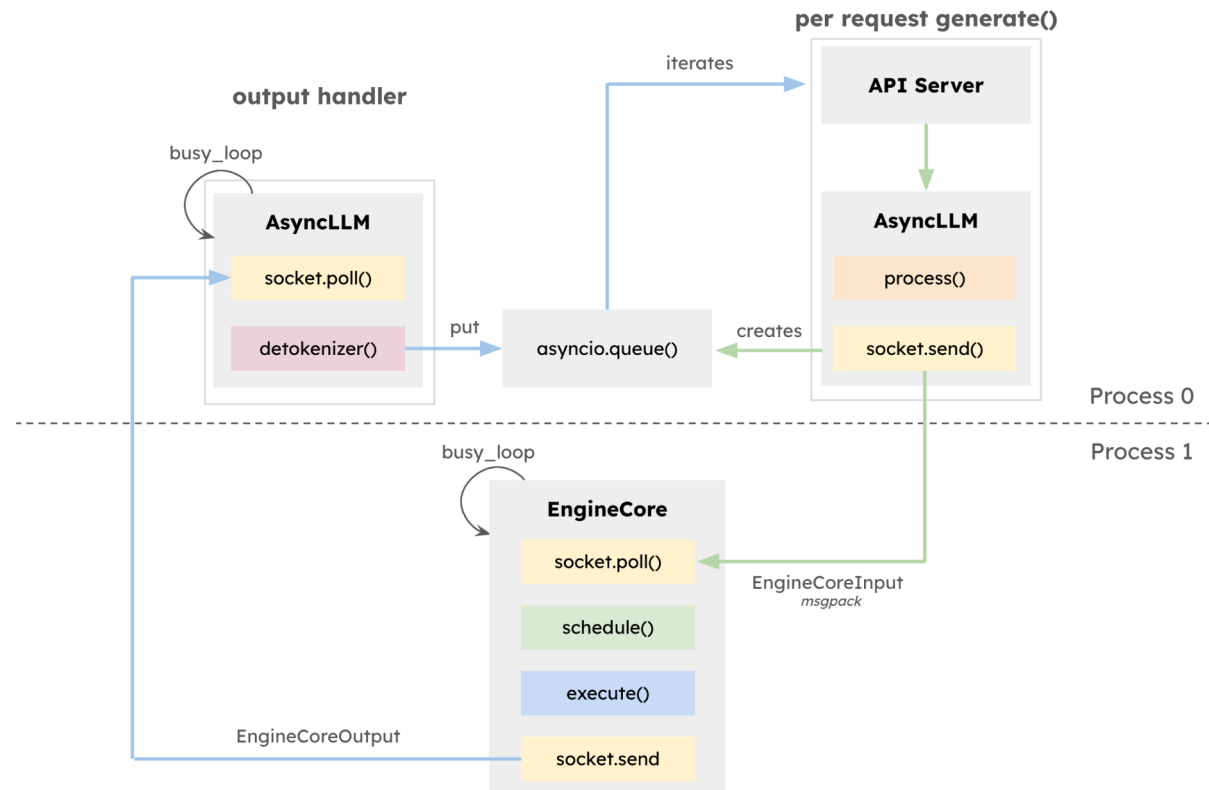
# Optimized Engine Loop & API Server

Goal: Make sure GPU is not stalled

- By pre-processing
  - E.g., converting JPEG images into input tensors (resizing, cropping, ...)
- By post-processing
  - E.g., de-tokenizing output token IDs into output strings
- By HTTP request handling
  - E.g., streaming outputs to 100s of concurrent users

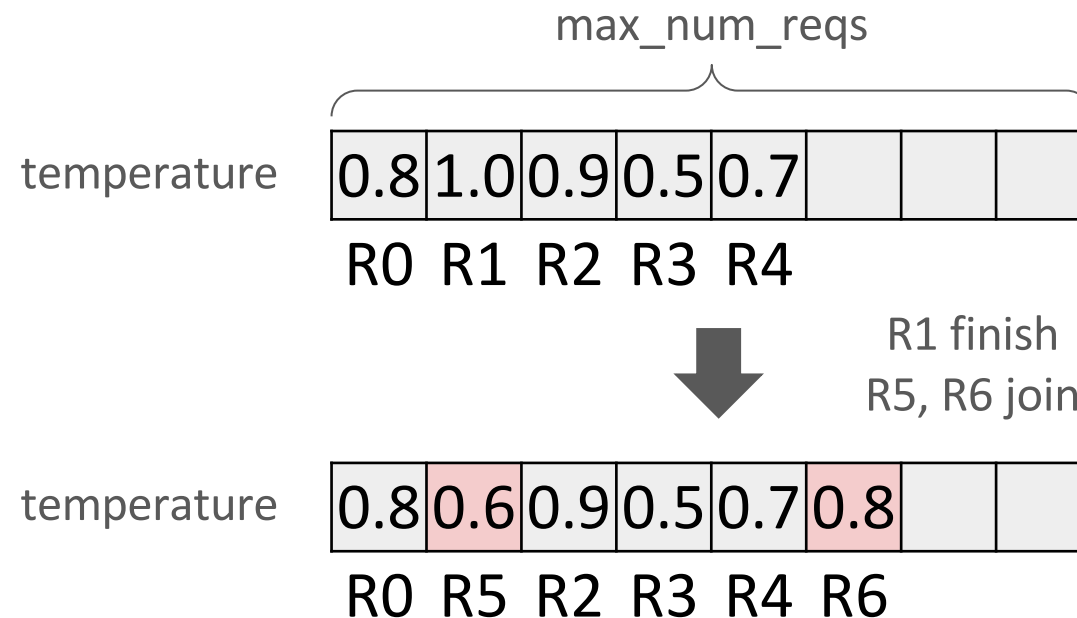
# Optimized Engine Loop & API Server (cont'd)

- **Two-process** approach
  - **Process 0 (Frontend):** Pre-/post-processing & API Server
    - Importantly, de-tokenization happens in Process 0
  - **Process 1 (EngineCore):** Schedule & execute the model every step
    - A busy loop that is NOT blocked by Process 0

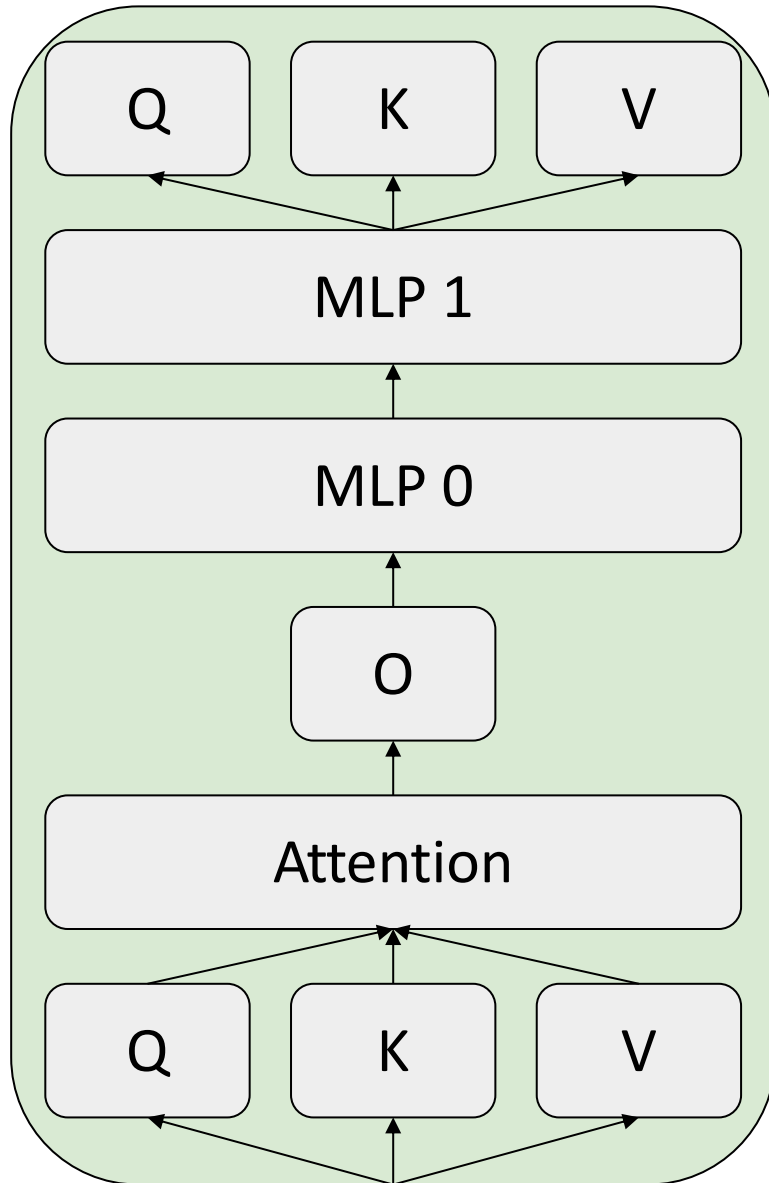


# Incremental Input Preparation (Persistent Batch)

- In V0, input tensors are re-created from scratch at every step
- In V1, **input tensors are cached**, and we **only apply the diff** at each step
  - Typically, the diff is minimal because only a few requests join or finish at each step
  - The gain is larger for larger objects like block table

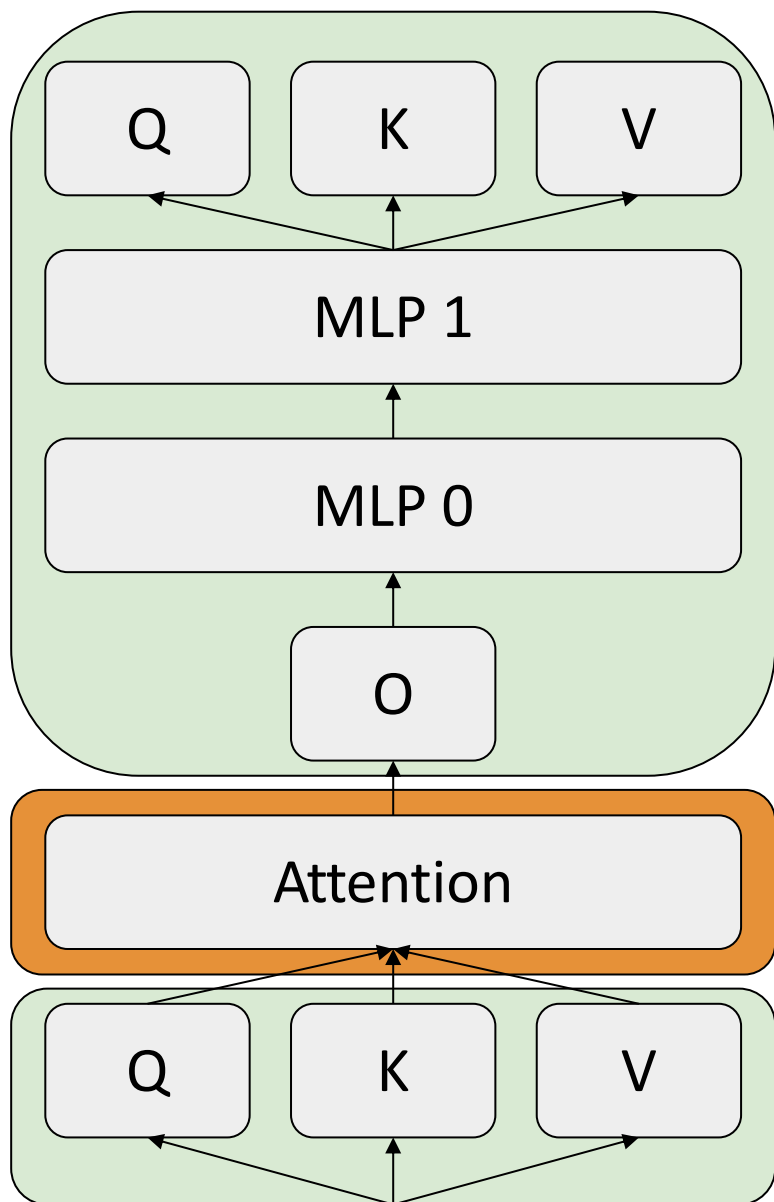


# Piecewise CUDA Graphs



- V0: Single CUDA graph for the entire model
- Pros: Minimal CPU overheads in model execution
- Cons: Limited flexibility
  - Static shapes are required
  - No CPU operations are allowed→ Increased development burden

## Piecewise CUDA Graphs (cont'd)



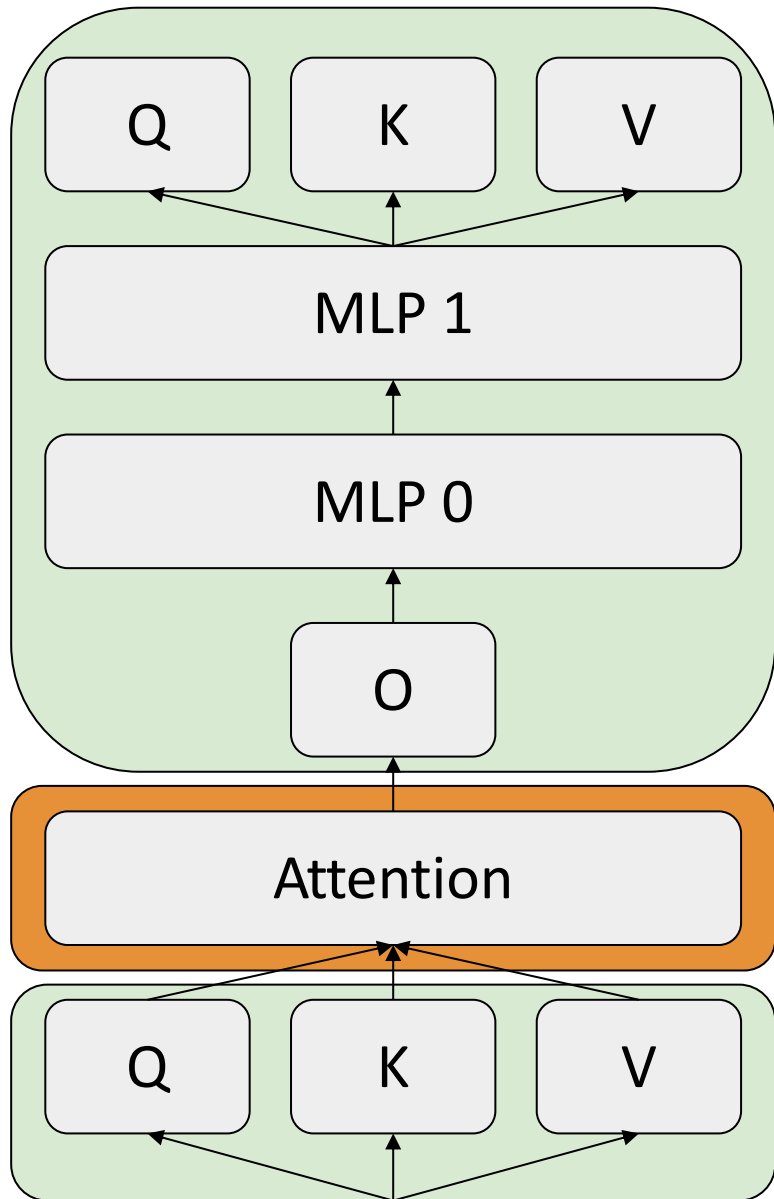
CUDA graph N

PyTorch Eager

CUDA graph N-1

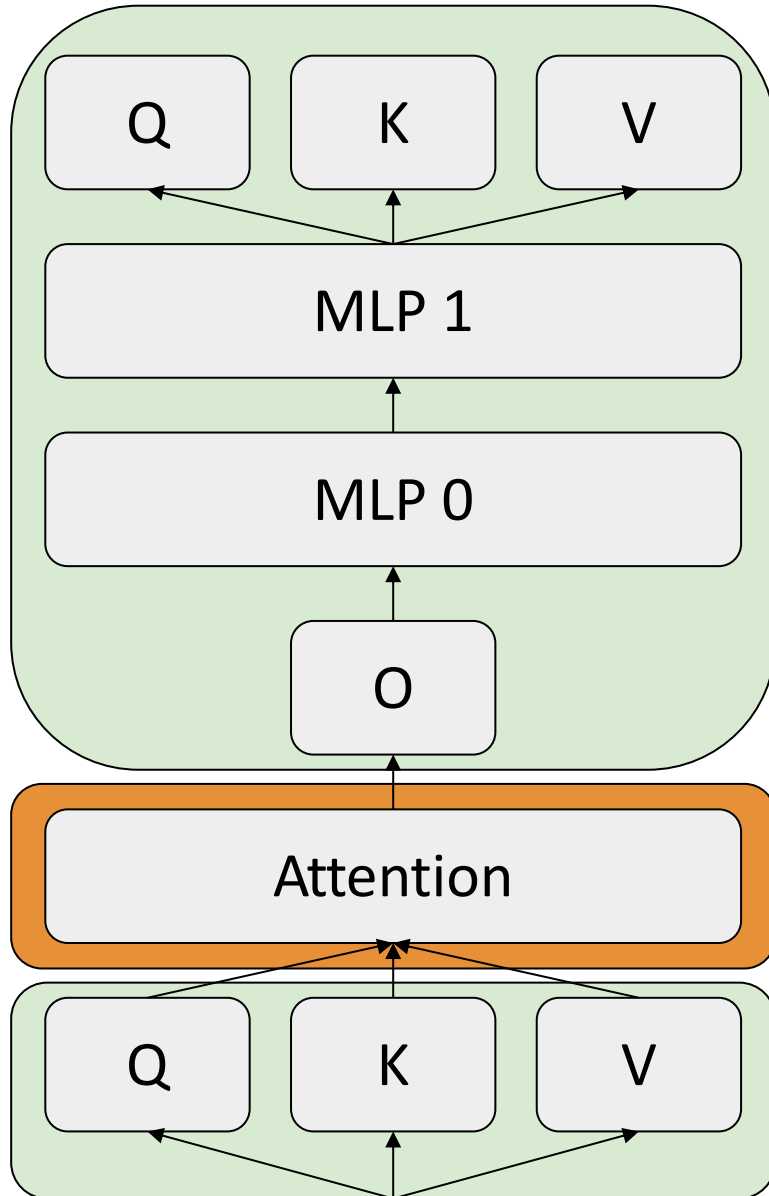
Graph split using  
[torch.compile](https://pytorch.org/docs/stable/torch.compile.html)

## Piecewise CUDA Graphs (cont'd)



- V1: Splits the model into pieces
  - Runs the attention op in **eager-mode PyTorch**
  - Runs other ops with **CUDA graphs**
    - Easy to capture, since the ops are token-wise
- Pros: **Maximum freedom** in implementing the attention op
  - No restriction on shapes
  - Any CPU operations are allowed
- Cons: **CPU overheads** unhidden by CUDA graphs could slow down the model execution
  - Negligible for 8B+ models on H100
  - Also supports full-cudagraph for certain attention backends (e.g. FA3)

# Piecewise CUDA Graphs (cont'd)



- The impact goes beyond inference
  - Training engine (Megatron) also starts to support piecewise cudagraph (per-layer cudagraph)

## NVIDIA Megatron Core 0.9.0

- Uneven pipeline parallelism
  - Enable pipeline parallelism where first and last ranks have fewer transformer layers than the intermediate ranks
- Per layer CUDAGraph support for GPT training with Transformer Engine modules
- Enable different TP sizes for the vision encoder
- Enable pipeline parallelism for T5 & Llava models
- Support multi-tile multi-image input in Llava models

- Good for researchers to explore new variants

### EVALUATING TRAINING-FREE SPARSE ATTENTION

Sparse Frontier + vLLM



50 lines and it works for 100 models

HF



I read 2k lines file, did 5 patches, and TP still doesn't work

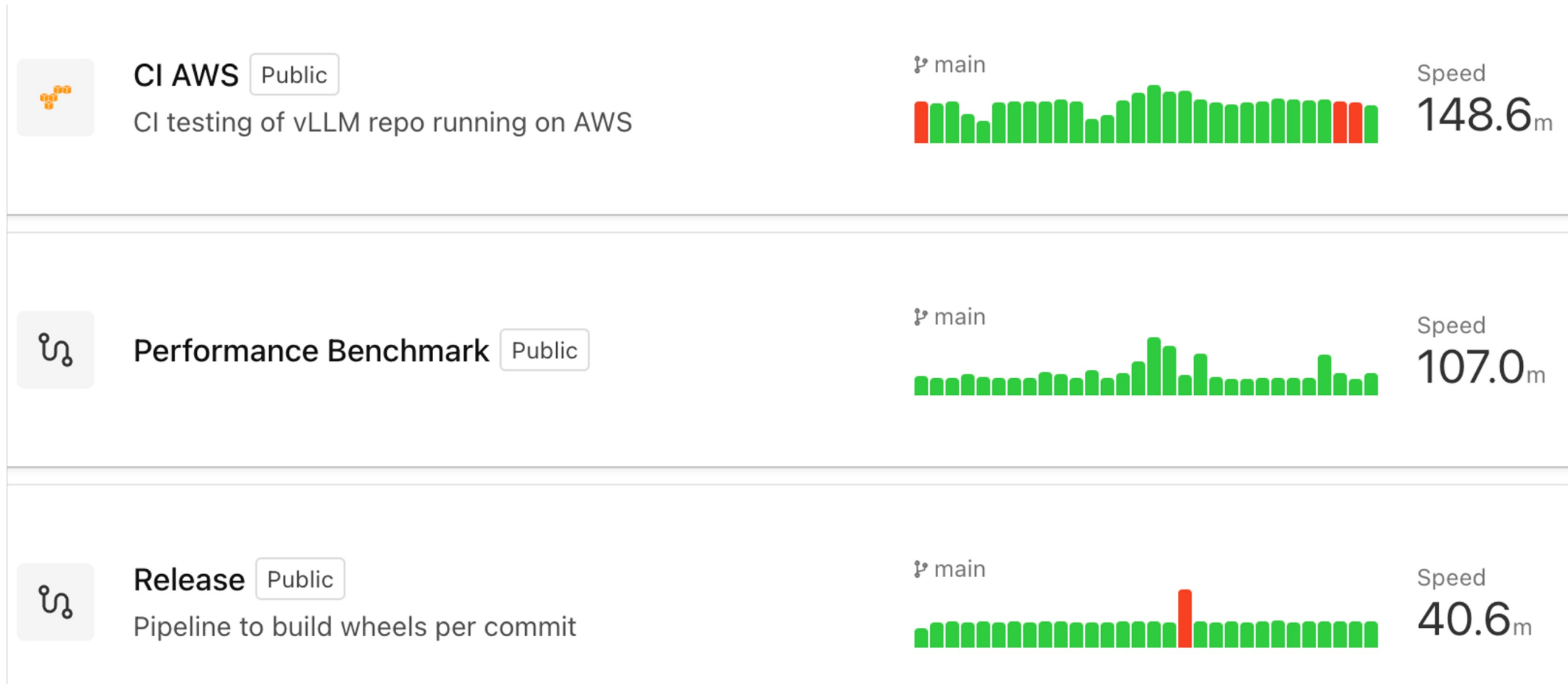
# Install and develop with ease

- For the latest release:
  - `$ pip install vllm`
  - Docker Hub: [vllm/vllm-openai](https://hub.docker.com/r/vllm/vllm-openai)
- Use vLLM with latest bug fixes:
  - Per-commit docker images and wheels
- Developing on top of vLLM
  - `$ VLLM_USE_PRECOMPILED=1 pip install -e .`
  - **No need to spent hours to compile cuda kernels!**
  - Check out the [doc](#) for a complete guide.

```
# 8xB200
# uv pip install -U --pre vllm \
#   --index-url https://wheels.vllm.ai/nightly \
#   --torch-backend=auto
vllm serve moonshotai/Kimi-K2-Instruct \
  --trust-remote-code \
  --tensor-parallel-size 8 \
  --enable-auto-tool-choice \
  --tool-call-parser kimi_k2
```

# CI/CD

50k\$ / month



Every commit is tested with thousands of tests

# Owned & Driven by Community

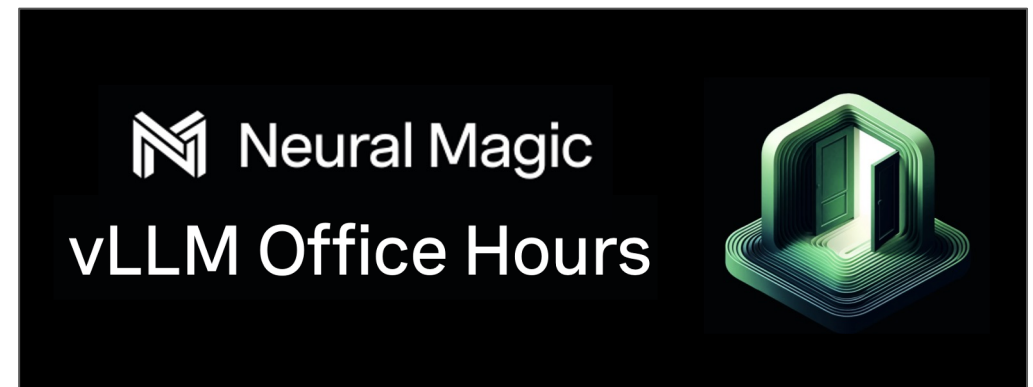
vLLM has joined [PyTorch Foundation](#)!



vLLM Meetups (every 2 months)



vLLM Office Hours (every 2 weeks)



# PyTorch + vLLM = ❤️

## Key takeaways:

- PyTorch and vLLM are both critical to the AI ecosystem and are increasingly being used together for cutting-edge generative AI applications, including inference, post-training, and agentic systems at scale.
- With the **shift** of the PyTorch Foundation to an umbrella foundation, we are excited to see projects being both used and supported by a wide range of customers, from hyperscalers to startups and everyone in between.
- vLLM is leveraging the broader PyTorch ecosystem to accelerate innovation, benefiting from projects such as torch.compile, TorchAO, FlexAttention, and collaborating to support heterogeneous hardware and complex parallelism.
- The teams (and others) are collaborating to build out PyTorch native support and integration for large-scale inference and post-training.

# PyTorch + vLLM = ❤️

vLLM tests failing in torch 2.8rc but passing with torch 2.7 #157461

Open

Edit

New issue



zou3519 opened last week · edited by zou3519

Edits

Contributor



- ☐ tests/basic\_correctness/test\_cumem.py::test\_end\_to\_end[meta-llama/Llama-3.2-1B-True]
- ☐ tests/basic\_correctness/test\_cumem.py::test\_end\_to\_end[facebook/opt-125m-False]
- ☐ entrypoints/openai/test\_sleep.py::test\_sleep\_mode
- ☐ entrypoints/openai/test\_translation\_validation.py::test\_long\_audio\_request
- ☐ TODO(rzou): there are more things that are failing, go list them out.

cc @ezyang @gchanan @kadeng @msaroufim

Create sub-issue



zou3519 added this to the 2.8.0 milestone last week

Assignees



malfet



zou3519

Labels

high priority

module: bc-breaking

module: regression

triaged

Type

No type

Projects

No projects

Milestone

2.8.0

Due by July 30, 2025, 73% complete

[test][distributed][vllm] stabilize the p2p sharing through ipc #158089

Edit

< > Code

Closed

youkaichao wants to merge 4 commits into pytorch:main from youkaichao:dist\_test



Conversation 9

Commits 4

Checks 161

Files changed 1

+78 -0



youkaichao commented 2 days ago ·

Collaborator



edited by pytorch-bot bot

vLLM's RLHF integration [https://github.com/vllm-project/vllm/blob/cf75cd2098f6a3f0bc38d92d1669810c084dab9b/examples/offline\\_inference/rlhf\\_utils.py#L93](https://github.com/vllm-project/vllm/blob/cf75cd2098f6a3f0bc38d92d1669810c084dab9b/examples/offline_inference/rlhf_utils.py#L93) depends on this hidden feature, adding the test so that PyTorch will not break it in a backward-incompatible way.

The goal is to create p2p shared tensors across devices, say sharing process 0's memory on GPU 0, to process 1's memory space on GPU 1, when GPU 0 and GPU 1 can use GPU direct p2p access.

cc @H-Huang @awgu @wanchao @fegin @fduwjj @wz337 @wconstab @d4l3k

Reviewers



ngimel



houseroad

Assignees

No one—assign yourself

Labels

ciflow/trunk

Merged

oncall: distributed

open source

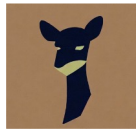
topic: not user facing

Projects

None yet

# LLM Adopters

## Open-Source Projects



lm-sys/FastChat



OPENCHAT



EleutherAI/lm-evaluation-harness



allenai/open-instruct



outlines-dev/outlines

...

## Companies



anyscale



Google Cloud



Microsoft

ORACLE

Cloud Infrastructure



snowflake



intel®



ROBLOX

Uber



databricks

LinkedIn



scale

...

# Productionized on Many Clouds!



**SageMaker** runtime for open ended generation.



Default choice powering **Azure AI Studio**.



**Vertex Model Garden** uses vLLM for Open LLMs.



Default template for serverless product!

## The Path to Open-Sourcing the DeepSeek Inference Engine

---

A few weeks ago, during [Open Source Week](#), we open-sourced several libraries. The response from the community has been incredibly positive - sparking inspiring collaborations, productive discussions, and valuable bug fixes. Encouraged by this, we've decided to take another step forward: contributing our internal inference engine back to the open-source community.

We are deeply grateful for the open-source ecosystem, without which our progress toward AGI would not be possible. Our training framework relies on [PyTorch](#), and our inference engine is built upon [vLLM](#), both of which have been instrumental in accelerating the training and deployment of DeepSeek models.

Given the growing demand for deploying models like [DeepSeek-V3](#) and [DeepSeek-R1](#), we want to give back to the community as much as we can. While we initially considered open-sourcing our full internal inference engine, we identified several challenges:

- **Codebase Divergence:** Our engine is based on an early fork of vLLM from over a year ago. Although structurally similar, we've heavily customized it for DeepSeek models, making it difficult to extend for broader use cases.
- **Infrastructure Dependencies:** The engine is tightly coupled with our internal infrastructure, including cluster management tools, making it impractical for public deployment without significant modifications.
- **Limited Maintenance Bandwidth:** As a small research team focused on developing better models, we lack bandwidth to maintain a large open-source project.

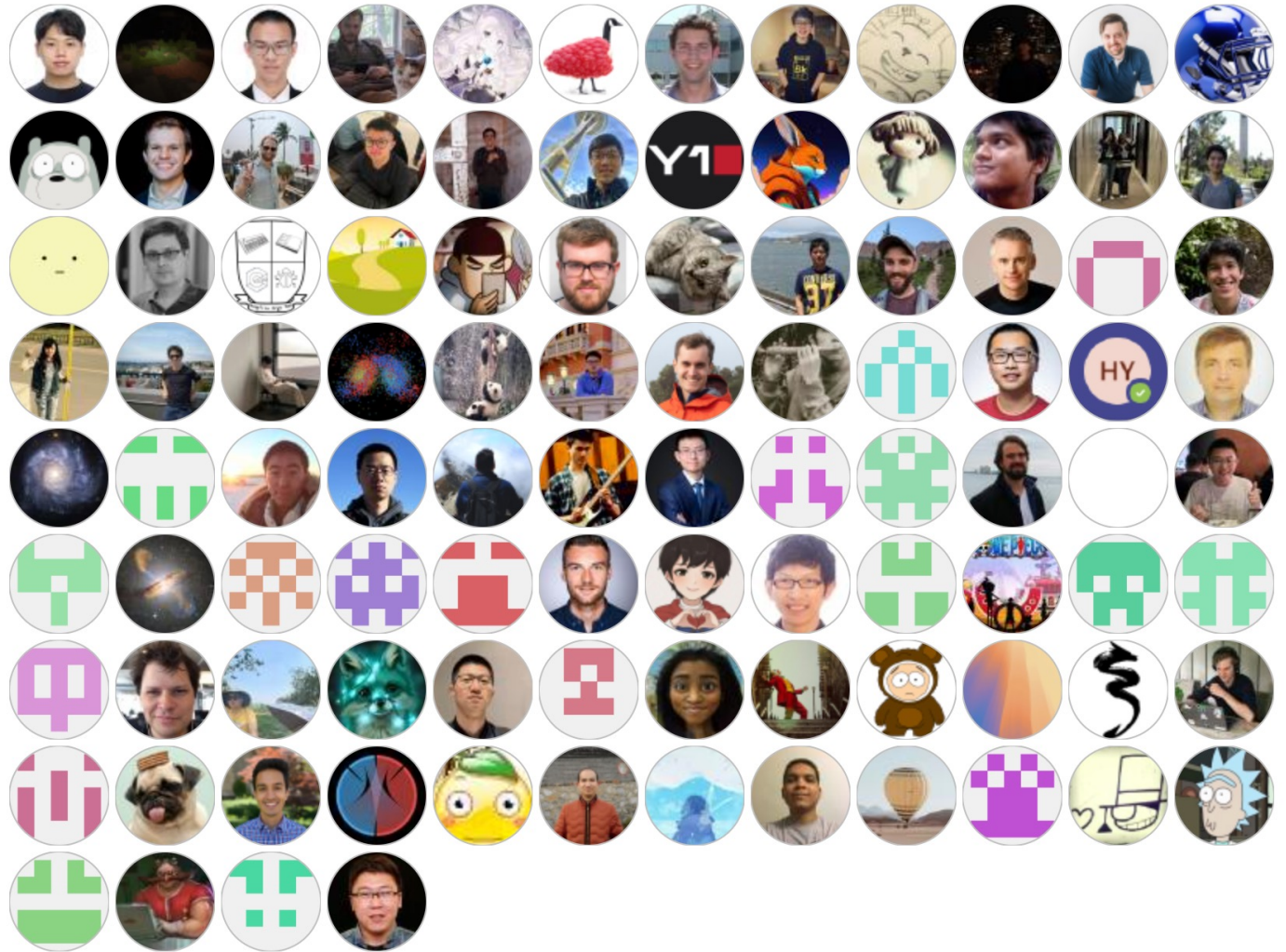
Considering these challenges, we've decided to collaborate with existing open-source projects as more sustainable alternatives.

Moving forward, we will work closely with existing open-source projects to:

- **Extract Standalone Features:** Modularize and contribute reusable components as independent libraries.
- **Share Optimizations:** Contribute design improvements and implementation details directly.

# Who develops vLLM?

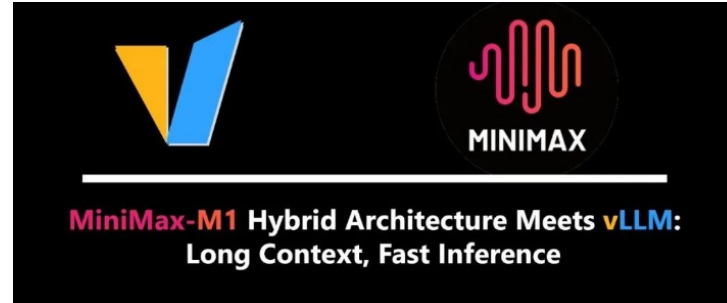
Thanks to 1200+  
**contributors** who  
raised issues,  
participated in  
discussions, and  
submitted PRs!



# vLLM: Building the ecosystem for LLM Inference



## Introducing vLLM Hardware Plugin, Best Practice from Ascend NPU



blog.vllm.ai



## Accelerating RLHF with vLLM, Best Practice from OpenRLHF



Thank you sponsors (funding compute!)



databricks



deepinfra



Dropbox



Google Cloud



Lambda



NEBIUS



novita.ai



NVIDIA

SEQUOIA



Replicate

ROBLOX



RunPod



Trainy



ZhenFund  
真格基金



Building the **fastest** and **easiest-to-use**  
open-source LLM inference & serving  
engine!



<https://github.com/vllm-project/vllm>



<https://slack.vllm.ai>



<https://docs.vllm.ai>



Ask AI



[https://twitter.com/vllm\\_project](https://twitter.com/vllm_project)



<https://blog.vllm.ai>



<https://opencollective.com/vllm>



<https://www.zhihu.com/people/vllm-team>



vllm\_project